

AMPS

Adaptive Mesh Particle Simulator

User and Developer Manual

Core, applications, physics, coupling and validation

AMPS development team

Repository documentation edition: 10 November 2026

This manual describes the source tree and bounded validation available at the edition date. A documented capability is not a claim that every combination of algorithms, inputs, machines or scientific applications has been qualified.

How to use this manual

Start with Chapters 1–4 for a first run. Choose the physics and boundary chapters before choosing numerical controls. Chapter 16 explains evidence and test execution; Chapter 18 provides executable examples with explicit oracles. Developers should also read Chapters 3, 15 and 19. Paths in code listings are relative to the AMPS root unless a different working directory is shown.

Evidence vocabulary. *Implemented* means code exists. *Verified* means a stated numerical or software contract has independent evidence. *Scientifically validated* requires an appropriate physical reference or observations, error estimates and the declared domain of validity. These terms are deliberately distinct. The test catalog lists registered intent; an actual run receipt records execution. Historical numerical results in this manual are dated snapshots.

Build independence. The manual compiles from a copy of this directory with a suitable TeX engine; neither an AMPS executable nor SWMF, SPICE, a generated build tree or remote figures are required. The accompanying README gives the exact commands.

Contents

How to use this manual **i**

1 Introduction and model scope **1**

1.1 What AMPS solves . . . 1
1.2 Choose the physical problem first . . . 1
1.3 Four kinds of evidence . . . 1
1.4 Reading and operating conventions . . . 2

2 Installation, configuration and execution **3**

2.1 Prerequisites and source distribution . . . 3
2.2 Site configuration and application selection . . . 3
2.3 Mandatory native rebuild sequence . . . 4
2.4 SPICE configuration and data readiness . . . 4
2.5 Run directories, MPI and batch execution . . . 4
2.6 A first verification before a science run . . . 5

3 Core and application architecture **6**

3.1 Layered architecture and data flow . . . 6
3.2 Ownership of responsibilities . . . 6
3.3 Initialization and the simulation loop . . . 7
3.4 Particles, lists and extensible storage . . . 7
3.5 Ownership, MPI and optional parallelism . . . 9
3.6 Public model libraries and application adapters . . . 9

4 Compile-time and runtime input **10**

4.1 Two layers with different lifetimes . . . 10
4.2 The generic runtime grammar . . . 10
4.3 Named cases and complete receipts . . . 11
4.4 Examples of real owner controls . . . 11
4.5 Source and observation files . . . 11
4.6 Validation and provenance rules . . . 12

5 Kinetic formulation and numerical controls **13**
5.1 Distribution functions and statistical particles . . . 13
5.2 Time step, mesh resolution and particle weight . . . 13
5.3 Choosing a discretization for DSMC . . . 14
5.4 Sampling, stationarity and uncertainty . . . 15
5.5 Random streams and population control . . . 15

6 Adaptive mesh, geometry and interpolation **16**
6.1 Tree, blocks and dimensionality . . . 16
6.2 Resolution functions and adaptation . . . 16
6.3 Interpolation contracts . . . 17
6.4 Cut cells and internal boundaries . . . 17
6.5 Distributed geometry and output . . . 17

7 Particle movers, forces and field solvers **18**
7.1 Neutral and charged characteristics . . . 18
7.2 Selecting a mover . . . 18
7.3 Electromagnetic PIC and ECSIM . . . 18
7.4 Boundary events and complete trajectories . . . 19

8 Collisions and internal degrees of freedom **20**
8.1 Binary collisions and the DSMC approximation . . . 20
8.2 Models, parameters and selection . . . 20
8.3 No-time-counter selection and unequal weights . . . 21
8.4 Rotational and vibrational energy . . . 21
8.5 Independent validation . . . 21

9 Photochemistry and other reactions **22**
9.1 Rates, channels and parent survival . . . 22
9.2 Daughters and event energetics . . . 22
9.3 Charge exchange and electron impact . . . 22
9.4 Networks, stiffness and weighted estimators . . . 23
9.5 Transactional production callbacks . . . 23
9.6 The Enceladus parent-loss scope . . . 23

10 Dust grains and grain populations **24**
10.1 Size is a physical state . . . 24
10.2 Size distributions and importance weights . . . 24
10.3 Charging and forces . . . 24
10.4 Dust-specific split and merge . . . 25

10.5 Spherical injection and its ledger . . . 25

11 Surfaces, arbitrary shapes and source boundaries 26

11.1 Geometry is separate from surface physics . . . 26
11.2 Diffuse emission and accommodation . . . 26
11.3 Exosphere source processes . . . 27
11.4 Importing an arbitrary shape . . . 27
11.5 Source positions and normalization . . . 27

12 Planetary, satellite and comet applications 28

12.1 A common kinetic engine, different physical targets . . . 28
12.2 The Moon: exosphere, terrain and observation geometry . . . 28
12.3 Enceladus: sources, parent loss and Cassini comparisons . . . 28
12.4 Comets: radial and shape-based gas/dust models . . . 29
12.5 Planetary energetic particles . . . 29

13 Heliospheric and energetic-particle physics 31

13.1 Field-line and three-dimensional formulations . . . 31
13.2 Mean free path, diffusion and turbulence . . . 31
13.3 Background providers and physical state . . . 32
13.4 CME implementation and qualification boundary . . . 32

14 SWMF and external background coupling 33

14.1 Component lifecycle . . . 33
14.2 Communicators and processor layout . . . 33
14.3 Parameter ownership and schedules . . . 34
14.4 Exchange contracts . . . 34
14.5 File providers, SPICE and empirical models . . . 34
14.6 Coupled testing and isolation . . . 34

15 Sampling, output, observations and restart 36

15.1 Data lifetime and normalization . . . 36
15.2 Application-defined node data and Tecplot . . . 36
15.3 Synthetic observations . . . 36
15.4 Persistent formats and restart identity . . . 37
15.5 Continuation and archive practice . . . 37

16 Verification, application tests and nightly reporting 38

16.1 What the global procedure does . . . 38
16.2 Required machine configuration . . . 38

16.3 Short, full and core-only commands . . . 39
16.4 Concurrency and native builds . . . 39
16.5 Exact cases, reporting and failures . . . 39
16.6 Independent oracles and qualification . . . 40
16.7 Adding a test . . . 40

17 Individual application regression tests **42**
17.1 The legacy application procedure and its authorities . . . 42
17.2 How Table records control generation . . . 42
17.3 Four phases and an individual invocation . . . 43
17.4 Application and numerical families . . . 44
17.5 References, tolerances and comparison artifacts . . . 45
17.6 Dependencies, preflight and concurrency . . . 45
17.7 Maintaining this documentation . . . 46

18 Examples that are also end-to-end tests **47**
18.1 How to interpret an example . . . 47
18.2 A 3-D kinetic application: motion, collisions and parent decay . . . 47
18.3 A daughter-producing photochemical mesh . . . 47
18.4 Surface sources and inventory in a native exosphere . . . 48
18.5 Dust charging and physical injection . . . 48
18.6 Adaptive geometry and actual MPI ownership . . . 48
18.7 Enceladus E3: a physical forward-model campaign . . . 49
18.8 Moon: regression and science qualification are separate . . . 49
18.9 Generic comet chemistry and brightness . . . 49
18.10 Legacy electromagnetic wave regression . . . 50
18.11 Further Table-controlled application examples . . . 50

19 Extending applications and maintaining the code **52**
19.1 Begin with a physical and software contract . . . 52
19.2 A practical new-application sequence . . . 52
19.3 Comments and documentation . . . 53
19.4 Portability and reproducibility . . . 53
19.5 Performance without changing the experiment . . . 53
19.6 Source and reference preservation . . . 53

20 Diagnostics and troubleshooting **55**
20.1 Start from the earliest failed boundary . . . 55
20.2 Native build identity and SPICE isolation . . . 56

20.3 Distinguish numerical error from missing physics . . . 56
20.4 Reporting a reproducible issue . . . 56

A Reference map, units and documentation authority **57**
A.1 Conventions . . . 57
A.2 Where to find the authoritative detail . . . 57
A.3 Supplementary legacy notes . . . 58
A.4 Maintaining the manual . . . 58

B Registered test-suite snapshot **59**

C Legacy application-regression catalog **65**
C.1 Heliospheric transport and coupled plasma . . . 65
C.2 Geospace cutoff, access and flux . . . 68
C.3 Planetary, comet and dust applications . . . 70
C.4 Numerical and process fixtures . . . 72
C.5 Fields, waves, solvers and periodic boundaries . . . 73
C.6 Individual trajectories and mover applications . . . 75
C.7 Declared but absent from the generated host snapshot . . . 77

Bibliography **79**

List of Tables

3.1 Repository map and responsibilities. 6

4.1 Input ownership and rebuild requirements. 10

16.1 Result vocabulary. 40

17.1 Application-regression declaration fields. 42
17.2 The 100 currently generated application owners. 44

20.1 Symptom-to-diagnosis guide. 55

A.1 Common dimensional quantities. 57
A.2 Maintained detailed contracts. 57

B.1 All registered core and application test owners. 59

C.1 Generated targets: Heliospheric transport and coupled plasma. 65
C.2 Generated targets: Geospace cutoff, access and flux. 69
C.3 Generated targets: Planetary, comet and dust applications. 70
C.4 Generated targets: Numerical and process fixtures. 72
C.5 Generated targets: Fields, waves, solvers and periodic boundaries. 73
C.6 Generated targets: Individual trajectories and mover applications. 75
C.7 Additional Table declarations and machine selectors. 77

Chapter 1

Introduction and model scope

1.1 What AMPS solves

AMPS is a kinetic simulation framework for rarefied gas, neutral and charged particle populations on adaptive meshes. Applications use its particle records, transport, stochastic reactions/collisions, surface interactions, sampling and distributed data structures. Depending on configuration, fields are prescribed, coupled from another model, or advanced by supported PIC solvers. The same framework supports planetary exospheres, comet comae and dust, magnetospheric particles and heliospheric transport.

AMPS can run standalone or as an SWMF component. A standalone application owns its outer lifecycle; SWMF supplies component communicators, scheduling and coupling epochs through wrappers. Shared numerical machinery does not make the scientific assumptions of one application valid in another.

1.2 Choose the physical problem first

A useful run description states the species, phase-space representation, geometry/frame, initial and external boundary conditions, injection rate and velocity law, force model, reaction network, surface physics and observable. Then state mesh, time-step, particle-weight and sampling policies. If any required physical information is absent, resolve it before running: numerical convenience is not a physical closure.

For example, a neutral Enceladus parent-loss calculation can have H2O and CO2 particles while allocating no photoproducts and using no plasma. A comet network can instead produce daughters with recoil and internal energy. A field-line SEP model and a full 3-D charged-particle mover have different transport approximations. A source-free analytical CME background test cannot qualify particle acceleration or a full downstream plasma volume.

1.3 Four kinds of evidence

1. Structural checks verify grammar, registration, data identity and supported configuration. They need not evolve any physical state.

2. Numerical verification compares an algorithm with independent equations, manufactured solu-

tions, conservation or convergence.

3. Regression testing detects change relative to reviewed fixtures and can include a full configure/build/run/output path.

4. Scientific validation compares the declared physical model with relevant laboratory, observational or independently justified physical reference data.

These layers complement each other. A model can conserve particles while using the wrong source law; it can reproduce a fitted observation without independently validating that source. Reports must preserve the distinction.

1.4 Reading and operating conventions

Bare dimensional values in modern application runtime keys generally use the SI units stated by the owner; older generators and external tables can have explicit different units. The applicable input parser is authoritative. A path shown in a command is relative to the repository root unless a working directory is declared. Placeholders must be supplied by the operator. The documented test commands use a required machine profile rather than hard-coded compiler/kernel paths. Native build examples require an idle checkout and a clean generated build as explained next.

Source guide: `src/README.md;src/pic/README.md;src/test/README.md`

Chapter 2

Installation, configuration and execution

2.1 Prerequisites and source distribution

Use an existing authorized checkout or source archive with its maintained shared libraries, inputs and test fixtures. Preserve the supplied directory layout. This edition does not prescribe an obsolete CVS host or silently retrieve missing model data. Record the source revision and local changes with each production build.

The native toolchain needs a compatible C++ compiler, make, Perl and MPI; dependency-light modern contracts commonly require C++17. Python 3 runs the global test procedure and many post-processors. A Fortran compiler and matching MPI interface are required for SWMF and selected linked modules. Optional capabilities can require CSPICE, external field libraries, numerical tables, matplotlib/NumPy or an accelerator toolchain. Inspect the chosen application and machine profile rather than assuming all dependencies are universal.

2.2 Site configuration and application selection

The established `Config.pl` workflow selects the application and site settings. It uses `input/NAME.input` first and falls back to `input/NAME.conf` only if the former does not exist. Initial installation and compiler selection are site-specific; inspect `./Config.pl -help` and the existing `Makefile.conf`. Preserve site libraries and settings. Do not edit `.amps.conf` directly.

For a new compile-time application selection:

```
./Config.pl -show
./Config.pl -application=enceladus
```

Changing the runtime .in file selects only among compiled capabilities; it cannot change species IDs, dimension or allocated record layout. Generated files in `build` or `srcTemp` are not the maintained place to implement a lasting fix.

2.3 Mandatory native rebuild sequence

Before *every* native AMPS rebuild, confirm the repository root and verify that no build or test simulation is using it. In the current workspace the root is `/home/vtenishe/Mars2/AMPS`. From that confirmed, idle root:

```
# Remove only the generated root build directory.
rm -rf -- build

# Regenerate the chosen application's configuration/hooks.
./Config.pl -application=enceladus

# Use the established top-level compilation workflow.
env MAKEFLAGS="-j16" make amps
```

Some application guides explicitly stage generator hooks before make:

```
./ampsConfig.pl -input enceladus.input -no-compile
```

Follow the selected adapter's established sequence. Do not remove source directories, site settings, libraries or another checkout. A focused standalone compile of a shared-model test is not a native AMPS build.

The global native test adapters perform and record this preflight under exclusive ownership. Prefer them for reproducible verification. Do not run a second manual Config.pl or make against their active checkout.

2.4 SPICE configuration and data readiness

SPICE-enabled applications require both the toolkit and appropriate kernels. Configure explicit local paths through the established workflow, for example:

```
./Config.pl -spice-path=/path/to/SPICE/cspice \
  -spice-kernels=/path/to/SPICE/Kernels
```

These paths are placeholders. A no-SPICE fixture selects `-spice-path=nospice`; a component-level SPICE-off input by itself may not remove an inherited global toolkit/link choice. Test actual state and orientation queries for the intended target, frame and epoch. The presence of a file with a plausible name is not proof of coverage. Record versions/hashes and load order; never substitute an unrelated mission ephemeris to make initialization succeed.

2.5 Run directories, MPI and batch execution

Use a fresh run directory and freeze the exact executable and inputs. Paths inside application files may resolve relative to the input file or the run directory; inspect the owner. Copy/include all dependent source distributions, event files and data receipts, preserving that relationship. A minimal launcher pattern, after preparing a complete run, is:

```
cd /path/to/prepared/run
mpiexec -n 4 ./amps -input run.in > run.log 2>&1
```

This is a launch pattern, not a universal CLI or rank recommendation. I3 uses `-i FILE -case N`; some legacy applications take no runtime file. Choose the MPI rank flag and launcher from the site's configuration. Allocate memory for blocks, ghosts, particles, node fields and output buffers; counts of CPUs alone do not establish that a case fits.

A batch script should set the working directory, requested ranks/threads, required modules and log destinations explicitly. It must retain the real process return code. Do not assume a shell pipeline to a logging command preserves the simulation's failure code. Use the scheduler allocation, not an arbitrary host-specific job example from an old manual.

2.6 A first verification before a science run

Run the fast global campaign and then the relevant native/end-to-end owner:

```
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --campaign smoke --output-dir test_output/first-smoke
```

For another host, supply its real machine profile. Read the final individual FAIL/ERROR/SKIP list and linked logs. Keep compiler/startup success separate from completed physical stepping and scientific qualification.

Source guide: `Config.pl;ampsConfig.pl;src/test/README.md;test/qualification-machine/README.md`

Chapter 3

Core and application architecture

3.1 Layered architecture and data flow

Figure 3.1 separates physical case ownership from reusable numerical services. Configuration selects compiled capabilities and the runtime case. Applications translate those choices into source, force, boundary and observation hooks; shared physical models supply explicit equations and accepted state. A standalone driver and an SWMF wrapper can invoke the same core through different outer lifecycles.

The mesh identifies physical cells and surfaces; particles carry statistical state and follow characteristics; process/field modules update the selected physics. Sampling uses completed physical state, parallel exchange publishes the correct owner generation, and restart preserves the state needed for continuation. None of these services can supply a missing application source law or a scientifically unspecified boundary condition. The file map and initialization sections below connect each box to its maintained source and lifecycle.

3.2 Ownership of responsibilities

AMPS supplies reusable numerical machinery. An application supplies its physical target, configuration, sources, boundary laws, background selection and scientific products. A generic core callback must not acquire an Enceladus source rate or a comet-specific species assumption.

Table 3.1: Repository map and responsibilities.

Path	Responsibility
`src/pic`	Particle records, movers, collisions, reactions, sampling, field algorithms, exchange, time stepping and persistence.
`src/meshAMR`	Adaptive tree, blocks/nodes, geometric search, interpolation, internal boundaries and cut-cell operations.
`src/general`	Containers, random streams, numerical tools, diagnostics and the header-only generic runtime-input reader.
`src/models`	Shared physical models with their own contracts, data and focused tests. Some libraries are independent of PIC/MPI.
`src/interface`	External-library and empirical-provider interfaces; distinct from the SWMF component bridge.
`srcInterface`	SWMF PT/PC Fortran wrappers and the C++ AMPS bridge.

Path	Responsibility
`srcMoon`, `srcEnceladus`	Lunar exosphere and Enceladus source/observation applications.
`srcHartley2`, `srcCG`	Generic radial/shape-based comet applications and comet gas/dust models.
`srcEarth`, `srcSEP`, `srcSEP3D`	Planetary energetic-particle and heliospheric transport applications; their supported paths differ.
`input`	Compile-time decks, parser scripts, shared data choices and selected runtime examples.
`src/test`, `test`	Global registry/runners and native test applications, machine profiles and fixtures, respectively.
`build`, `srcTemp`	Generated compilation/preprocessing products, not a second source authority.
`doc`	This self-contained manual and its build scripts.

3.3 Initialization and the simulation loop

The exact entry point is application-owned, but the essential dependency order is stable:

1. Resolve and validate configuration, external data and compile-time capabilities. Reject an unsupported combination before a costly run.
2. Establish the correct MPI communicator and initialize the PIC core.
3. Register particle fields, node data, source and output callbacks.
4. Freeze associated-data offsets before allocating mesh blocks.
5. Build/read the mesh, assign ownership and allocate physical storage.
6. Initialize physical cell state, weights and time steps; publish owner state to ghosts before interpolation or output.
7. Inject, advance, exchange and apply the selected reaction/collision operators in the configured driver order; complete any continuation stages.
8. Sample completed physical state, finish output windows and checkpoint.
9. Finalize application products and MPI through the owning driver.

This is a dependency description, not a universal operator-splitting recipe. Inspect `src/pic/pic_time_step.cpp` and the selected application driver when establishing the order of collisions, chemistry, sampling or callbacks. An event inside a trajectory must not be processed again by a whole-step reaction callback.

3.4 Particles, lists and extensible storage

A model particle has a compiled species, position and velocity and may carry individual weight corrections, internal energy, dust attributes, trajectory metadata and application extension bytes. The byte layout is fixed by configuration. Use public record accessors rather than assuming offsets. Cloning

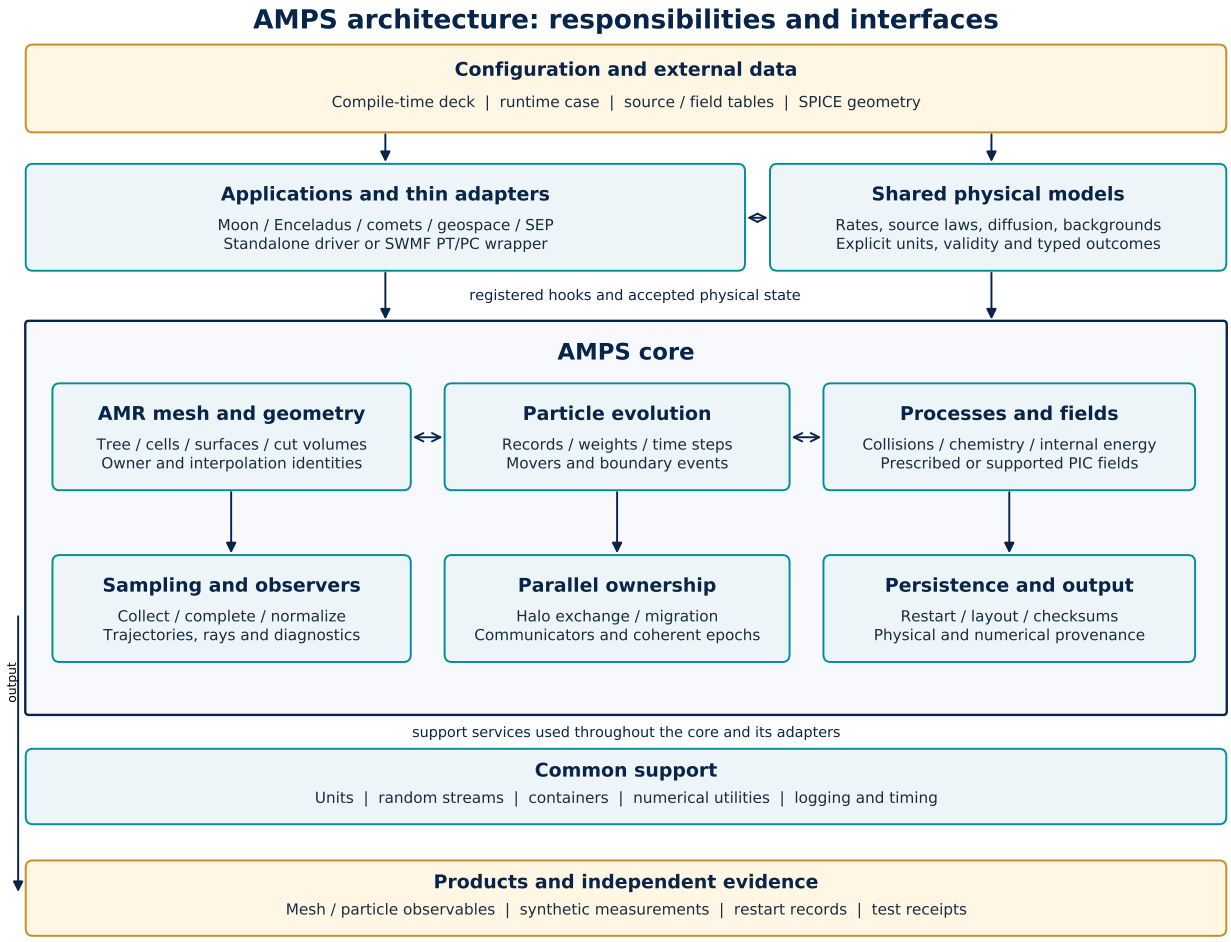


Figure 3.1: AMPS responsibility architecture. Arrows denote interfaces and state dependencies, not a fixed order of collision, chemistry or transport operators. Application adapters connect target physics to the core; shared physical libraries keep their own units, validity and test contracts. Ownership/exchange, sampling and persistence are separate responsibilities. Common utilities support all layers; test receipts are evidence about those layers rather than an additional physical evolution stage.

a record and inserting it into a linked list are separate operations; copying intrusive-list links into an already linked particle corrupts ownership. Deletion must unlink the exact record and return its storage once.

Cell-associated data have different lifetimes. Persistent background fields use static storage; accumulated moments use collecting/completed sampling buffers; derived display quantities may need no additional storage. Reserve all fields before `PIC::Mesh::initCellSamplingDataBuffer()`. Node buffers are byte-addressed: an application should use alignment-safe access, for example `std::memcpy`, when it cannot establish native alignment.

3.5 Ownership, MPI and optional parallelism

The replicated AMR topology is not the same as allocated physical blocks. A node can be known on a rank without its owner data being available there. Only the owning rank updates authoritative particle/cell state. Ghost exchange, migration and shared output must use the same completed generation. Do not replace a supplied component communicator with `MPI_COMM_WORLD`. A test that executes assertions only on rank zero does not verify remote-owner or received-ghost correctness.

MPI, OpenMP and accelerator support are configuration-dependent. A host routine existing in source does not prove that its callbacks or diagnostics are thread-safe or device-enabled. Transactional photochemistry and some correction-aware diagnostics explicitly reject unsupported hybrid modes. Treat that rejection as part of the API rather than removing the check.

3.6 Public model libraries and application adapters

Keep geometry-independent rates, units, reaction networks and transport relations in shared models. Thin adapters translate application configuration and accepted model state into PIC storage or callbacks. Public analytical libraries intended for independent use must not require application, PIC or MPI headers. Their tests should compile a neutral consumer and independently grade equations and failure behavior.

Source guide: `src/README.md;src/pic/README.md;src/test/README.md`

Chapter 4

Compile-time and runtime input

4.1 Two layers with different lifetimes

Compile-time decks use the AMPS generator vocabulary: main/general/module blocks, species topology, source directories, field/particle layouts, module selectors and generated hooks. Runtime input sets the selected physical case within those compiled capabilities. An input file ending in .input is not the same language or lifetime as a generic runtime .in file.

Table 4.1: Input ownership and rebuild requirements.

Layer	Typical choices	Rebuild?
Compile time	Species, dimension, module availability, block/ghost layout, particle extension capacity	Clean native rebuild.
Runtime owner	Source rate/shape, epoch, supported reactions, numerical controls, observers	New run; no rebuild if capabilities suffice.
External source/event file	Geometry, per-source strengths, kinematics, model identity and provenance	New run and validated receipt.
Machine profile	Compiler/launcher argv, data/output paths, capabilities	Test setup; not a physical source law.

4.2 The generic runtime grammar

The header-only reader in `src/general/input/amps_input.h` implements nestable `##begin KIND[: NAME]` and `##end KIND` blocks, `key = value` assignments, exclamation comments, trailing-backslash line continuation and `##include "PATH"`. Includes resolve relative to the including file and are cycle/depth checked. This grammar parses structure; each owner validates the meaning, units, required fields and supported values.

```
#begin application: example
  source.temperature_k = 175 ! illustrative owner-specific keys
  #begin case: quiet
    source.rate_s-1 = 8.0e27
  #end case
```

```
  #begin case: active
    source.rate_s-1 = 2.8e29
  #end case
#end application
```

This demonstrates grammar only; there is no generic application named example installed by this manual. Owners reject unknown keys rather than ignoring misspellings. Keys are case-sensitive; structural block-kind matching follows the generic reader's normalization. Diagnostics retain the originating file and line, including included files.

4.3 Named cases and complete receipts

A direct named case overlays common assignments and recursively matching non-case child blocks. Cases cannot be nested. When cases exist, selection must be explicit; an absent/unknown selector is an error. A selector for an owner with no cases is also rejected. Only the selected case survives into the resolved configuration and its fingerprint.

The generic reader does not prescribe one CLI spelling. Moon uses `-input`; Enceladus supports `-input` and `-input`; I3 uses `-i` and `-case`; current SEP owners document their aliases/default `amps.in`. Follow the application CLI, not an invented universal flag.

4.4 Examples of real owner controls

The maintained Enceladus receipt contains:

```
#begin application: enceladus
schema_version=2
source_distribution_file=enceladus_e3_equal_h2o_co2_sources.in
simulated_species=H2O,CO2
injected_species=H2O,CO2
chemistry_species=H2O
reaction_products=discard
primary_macroparticles_per_step.H2O=100
primary_macroparticles_per_step.CO2=100
characteristic_speed.H2O_m_s-1=300
characteristic_speed.CO2_m_s-1=300
courant.H2O=0.5
courant.CO2=0.5
sample_length_steps=1000
total_iterations=10000
particle_splitting_merging=off
#end application
```

This is an excerpt: it omits other required controls and is not a complete run file. Use `input/enceladus.in` and its validated companion source receipt. Its source speeds and thermal law are supplied separately; the characteristic speed here controls the numerical timestep, not source fitting.

4.5 Source and observation files

The same Enceladus source grammar supports shape and absolute mode. Shape mode requires a user total per species and normalizes nonnegative component strengths. Absolute mode forbids a con-

tradictory additional total. The file also names source distribution, frame, stripe projection, launch kinematics and all canonical components. A missing strength is an error, not an implicit zero. Documented future distribution identifiers must not be accepted before their geometry and source laws are implemented.

Repeated named observation blocks select data products. Adding an event requires its data reduction/response, epoch/frame, trajectory or ray geometry, units and comparison operator. Comment the complete block to omit it; commenting only its first line leaves an invalid document. Data paths belong in the run receipt or machine data setup, not hidden compiled defaults.

4.6 Validation and provenance rules

Validate finite values, physical ranges and dependencies before allocation. Do not accept an inactive control whose meaning is undefined merely because it is not currently used. Preserve the expanded/-canonical text, selected case, external-file hashes and compiled topology. The same-looking key in a retired INI deck is not current generic syntax; use maintained converters where provided, then validate the converted file.

Source guide: `src/INPUT_FILE.md;input/enceladus.in;srcEnceladus/SOURCE_DISTRIBUTION_FILE.md`

Chapter 5

Kinetic formulation and numerical controls

5.1 Distribution functions and statistical particles

For species s, let $f_s(\boldsymbol{x}, \boldsymbol{v}, t)$ be number density in phase space. A representative kinetic equation is

$$\frac{\partial f_s}{\partial t} + \boldsymbol{v} \cdot \nabla_x f_s + \frac{\boldsymbol{F}_s}{m_s} \cdot \nabla_v f_s = C_s[f] + S_s - L_s. \tag{5.1}$$

Which terms are present depends on the application. Neutral DSMC, test-particle transport and self-consistent electromagnetic PIC use different closures; they are not interchangeable run modes obtained by changing one rate.

A macroparticle p represents $w_p = W_{s,b} c_p$ physical particles, where $W_{s,b}$ is the selected base weight (possibly block-local) and $c_p > 0$ is an enabled individual correction. Fixed-weight configurations use the corresponding compiled convention. A weight is a physical multiplicity, not a particle mass and not an arbitrary plot normalization.

In cell volume V, instantaneous estimators are

$$n_s = \frac{1}{V} \sum_{p \in s} w_p, \qquad \boldsymbol{u}_s = \frac{\sum w_p \boldsymbol{v}_p}{\sum w_p}, \tag{5.2}$$

$$\mathsf{P}_s = \frac{1}{V} \sum_{p \in s} w_p m_s (\boldsymbol{v}_p - \boldsymbol{u}_s)(\boldsymbol{v}_p - \boldsymbol{u}_s)^T, \qquad T_s = \frac{\operatorname{tr} \mathsf{P}_s}{3 n_s k_B}. \tag{5.3}$$

The three-dimensional temperature formula applies to three translational velocity degrees of freedom. A DIM1 spatial mesh does not automatically remove transverse velocity components. A surface flux estimator instead uses crossing weight divided by area and physical accumulation time.

5.2 Time step, mesh resolution and particle weight

A common transport constraint for a global step is

$$\Delta t \le \min_{s,b,d} \frac{C_s h_{b,d}}{v_{\mathrm{char},s}}, \qquad 0 < C_s \le 1, \tag{5.4}$$

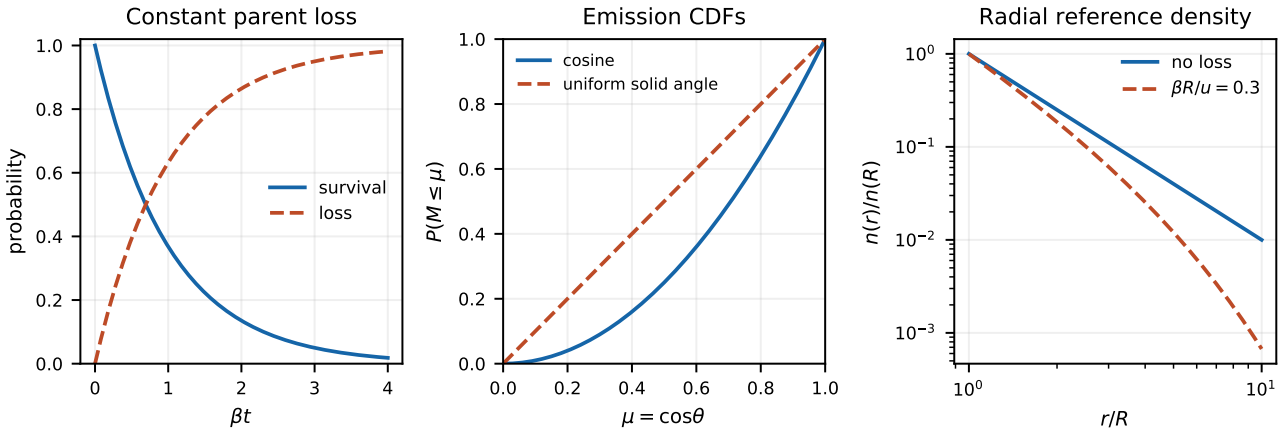


Figure 5.1: Independent analytical limits, not simulation results. Left: constant first-order parent loss. Centre: outgoing cosine-law and uniform-solid-angle CDFs in the normal-direction cosine. Right: steady spherical constant-speed outflow, $n(r)/n(R) = (R/r)^2 \exp[-\beta(r-R)/u]$, excluding gravity, collisions, returning particles and distributed sources. These limits illustrate different normalization and physical assumptions; they are not universal coma solutions.

with cell lengths $h_{b,d}$ in metres and speed in metres per second. This is an application-selected characteristic-speed bound, not a guarantee against the tail of a Maxwellian. Gyromotion, field variation, chemistry and collision accuracy can impose stricter limits. Log actual minimum cells and the final accepted step, not just the requested resolution.

For a constant physical source Q_s and an expected N_s numerical injections per step, $W_s = Q_s \Delta t / N_s$ when all particles use one weight. For a source with acceptance/rejection, local time steps or importance sampling, derive the represented *accepted* rate and its correction instead of applying that expression blindly. For local face transfer the represented crossing rate is $Wc/\Delta t$. The opt-in resampling multiplicity is

$$\mu_{\text{face}} = \frac{W_{\text{old}} \Delta t_{\text{new}}}{W_{\text{new}} \Delta t_{\text{old}}}. \tag{5.5}$$

The individual correction cancels only under the implemented policy of holding that correction fixed. Continuation across MPI owners must finish before sampling or generic splitting can duplicate an unfinished trajectory.

5.3 Choosing a discretization for DSMC

Resolve the shortest relevant density/temperature/flow scale and assess cell size relative to mean free path. Separate transport and collision time errors. For a dilute single-species estimate, $\lambda \sim 1/(\sqrt{2} n \sigma)$; mixtures and speed-dependent cross sections require an appropriate collision integral. Refine where collisional and surface gradients are strongest, then demonstrate convergence of the actual observable. Increasing particles alone does not remove mesh bias.

Representative counts per collision cell must support the selected pair algorithm. Sparse cells can have biased or noisy rates even when the total particle population is large. A large sample window does not remedy a systematic collision-cell or trajectory error.

5.4 Sampling, stationarity and uncertainty

Discard startup transients based on relaxation/transport times and observable histories, not an unexplained iteration count. A sampling length in steps corresponds to $N_{\text{sample}}\Delta t$ only for a fixed global step. Retain the actual physical accumulation interval for variable-step estimators. Estimate uncertainty using independent realizations or block averages long enough to expose autocorrelation. The familiar $N^{-1/2}$ statistical scaling does not justify treating correlated stored samples as independent.

Separate three studies: mesh refinement at controlled statistics, timestep refinement at controlled mesh, and particle/sample refinement at fixed discretization. Include boundary-distance and external-data resolution studies when those control the observable. Never fit a source with one numerical error and interpret the compensation as a measured physical rate.

5.5 Random streams and population control

Record seed, rank/thread layout and stream policy. Reproducibility on the same layout does not imply identical particles after repartitioning. For a constant Poisson rate ν, sample $\tau = -\log(1-u)/\nu$ with the generator's declared endpoint convention. Numerically `-log1p(-u)` resolves small u well; replacing the draw by `-log(u)` changes a fixed seeded path even though both have the same continuous distribution.

Generic split/merge must preserve the moments required by the application, and must specify what happens to opaque extension bytes. Conserving number, momentum and scalar energy does not preserve all angular or higher moments. Dust requires its own size-sensitive rules (Chapter 10).

Source guide: `src/pic/README.md;test/srcCoreUnitTests/README.md`

Chapter 6

Adaptive mesh, geometry and interpolation

6.1 Tree, blocks and dimensionality

AMPS uses an adaptive tree with 2^D children per refined node in represented dimension D. Leaves identify physical mesh blocks; blocks contain configured cell and ghost-cell counts. Tree topology, allocated blocks, particle lists and cell-associated fields are different objects with different ownership. DIM1 radial geometry additionally requires spherical measures; it is not equivalent to a Cartesian line with unit cross section.

For a radial spherical shell, the exact volume per steradian is

$$V_\Omega = \frac{r_R^3 - r_L^3}{3} = \frac{(r_R - r_L)(r_R^2 + r_R r_L + r_L^2)}{3}. \tag{6.1}$$

The factored form avoids subtraction of nearly equal large powers. Use a consistent volume centroid and surface measure in sampling, source and transport operators. A visually plausible radial grid with Cartesian normalization produces a physically incorrect density.

6.2 Resolution functions and adaptation

An application resolution function requests scales; the realized dyadic mesh is what controls transport and resource use. A flyby or limb ray can require refinement away from a surface. Refine the comparison region explicitly and record actual cells intersecting each observation, rather than relying on a small surface-cell size to resolve an entire trajectory.

Adaptation requires transfer policies for persistent state, sampled moments and particle ownership. Coarsening must not simply average quantities whose physical measure differs. Distinguish conservative extensive quantities from primitive intensive state. A changed mesh invalidates cached stencils and owner mappings. Verify generation changes and restored connectivity through refine/coarsen cycles and mesh-file round trips.

6.3 Interpolation contracts

AMPS provides legacy pointer stencils and decomposition-independent row stencils. A row identifies physical owner cells even if their blocks are not allocated on the requesting rank. This is useful for exchanged fields and distributed output; it does not make remote values locally accessible without an exchange.

Constant fields should be reproduced exactly within rounding error. A linear Cartesian field $g = 1.25 + 2x - 3y + 0.5z$ is an independent trilinear interpolation oracle. At a coarse/fine interface, verify support, nonnegative weights, unit sum and affine reproduction. A sum-to-one check alone cannot detect a geometrically wrong stencil. Test every applicable face, edge and corner orientation, boundary truncation, periodic wrapping and cache invalidation.

6.4 Cut cells and internal boundaries

A triangulated surface intersects cells and modifies accessible physical volume. Surface interception and volume integration are distinct numerical tasks. A mover can correctly hit a triangle while a cell-density normalization still uses an incorrect accessible volume. Very small cut cells require a defined normalization/time-step policy; do not silently replace their volumes with a floor without changing and validating the model.

Independent geometry controls include a planar cut of known volume, translated spheres, triangle areas/normals, tangent rays and convergence of area/volume with geometric resolution. Mesh topology and triangle indexing must survive reading, writing and reloading sparse external IDs.

6.5 Distributed geometry and output

Halo exchanges must transfer the correct fields for the current mesh and sampling generation. Mesh serialization and restart traverse 2^D children, not a hard-coded eight in lower dimensions. DIM1/DIM2 stored face caches still need valid initialization if a persistent mesh format reads them.

Geometry-only Tecplot output does not validate physical field callbacks. Whole-brick data output does not automatically clip an internal sphere. Select the cut-cell-aware distributed writer when clipped boundary geometry is required, and validate both geometry and data ordering.

Source guide: `test/srcCoreMeshTests/README.md;test/srcCoreMPITests/README.md;src/README.md`

Chapter 7

Particle movers, forces and field solvers

7.1 Neutral and charged characteristics

For a neutral under a prescribed force, $\dot{\boldsymbol{x}} = \boldsymbol{v}$ and $\dot{\boldsymbol{v}} = \boldsymbol{F}/m$. Gravity from a spherical body is $-\mathcal{G}M\,\boldsymbol{r}/r^3$ outside its mass distribution, with $\mathcal{G}M$ in $\mathrm{m}^3\,\mathrm{s}^{-2}$. Nonspherical gravity requires a separately defined mass model; a surface shape alone does not determine it. Radiation pressure and gas drag are distinct accelerations with their own size, velocity and geometry dependence.

Charged-particle trajectories satisfy

$$\dot{\boldsymbol{v}} = \frac{q}{m}(\boldsymbol{E} + \boldsymbol{v} \times \boldsymbol{B}) + \boldsymbol{a}_{\mathrm{other}}. \tag{7.1}$$

The relativistic equation advances momentum $\boldsymbol{p} = \gamma m \boldsymbol{v}$ with $\gamma = \sqrt{1 + p^2/(m^2c^2)}$; replacing m by a constant guessed relativistic mass is generally incorrect. Validate gyrofrequency, phase, parallel acceleration and drift against independent solutions.

7.2 Selecting a mover

The ordinary host dispatch is `PIC::Mover::DispatchParticleMover(ptr, dt, node)`. Legacy macro selection remains the default. Pointer selection requires its compiled legacy-settings switch to be off and a valid `PIC::Mover::SetUserDefinedParticleMover` callback installed before advancing a particle. Boundary-specific or device bulk kernels can have separate selection paths. The callback owns deletion/list insertion/migration according to the configured mover contract.

The timestep must resolve the fastest relevant gyroperiod and field scale, as well as cell crossings. Guiding-center motion assumes gyroaveraging is valid: gyroradius and gyroperiod must be small relative to background scales. Its drifts, mirror force and parallel electric force are not a valid fallback at a magnetic null; the field-local API provides typed invalid-domain status.

7.3 Electromagnetic PIC and ECSIM

Self-consistent PIC deposits particle charge/current and solves fields rather than reading a prescribed background. ECSIM is an energy-conserving semi-implicit formulation in the supported configuration. Verify the actual spatial operators, mass/moment assembly, boundary rows, solver residuals and particle-field energy accounting. Stability of a semi-implicit method does not remove resolution requirements for the phenomenon of interest.

For deposited shape weights S_i, require the intended partition of unity and $\sum_i Q_i = qw$ for one particle. Current conservation additionally requires a compatible discrete continuity equation, not just a correct total charge. Manufactured field tests establish consistency of operators on the tested meshes; they are not a blanket validation of every production plasma.

7.4 Boundary events and complete trajectories

Find the earliest physical boundary intersection within the remaining trajectory interval. Reflect or absorb at that event, then process the remaining time exactly once. A particle reaching a remote block must be delivered to its owner even if its continuation marker is already zero. Repeated reflection, tangency and starts on/inside a boundary need explicit tolerance and ownership conventions.

Use independent straight-line, constant-acceleration, uniform-field gyro, moving-plane and moving-sphere solutions as test oracles. Check phase-space invariants, event time, list membership, endpoint and represented losses; checking only final radius misses wrong velocity and duplicate ownership.

Source guide: `src/pic/README.md;test/srcCoreUnitTests/README.md;src/test/tests/pic/ecsim_operators/README.md`

Chapter 8

Collisions and internal degrees of freedom

8.1 Binary collisions and the DSMC approximation

DSMC separates molecular free flight from stochastic collisions over a short physical interval [1]. Pair selection is local to a collision cell. For species densities n_a, n_b, the physical cross-species rate per volume is

$$R_{ab} = n_a n_b \langle \sigma_{ab}(g) g \rangle, \qquad g = |\boldsymbol{v}_a - \boldsymbol{v}_b|. \tag{8.1}$$

For identical species the unordered-pair rate contains a factor 1/2. Finite populations require the appropriate pair-count correction; sampling the same numerical particle twice is not a physical self collision.

Elastic scattering preserves pair momentum and relative kinetic energy. Use the mass-weighted centre velocity and reduced mass:

$$\boldsymbol{V} = \frac{m_a \boldsymbol{v}_a + m_b \boldsymbol{v}_b}{m_a + m_b}, \qquad E_{\mathrm{rel}} = \tfrac{1}{2} \frac{m_a m_b}{m_a + m_b} g^2. \tag{8.2}$$

An isotropic hard-sphere angular law and a VSS angular law have different transport properties even when their total cross sections agree.

8.2 Models, parameters and selection

Hard-sphere (HS), variable-hard-sphere (VHS) and variable-soft-sphere (VSS) selectors are tested in separate compiled profiles. HS uses a constant total cross section. VHS introduces a speed dependence associated with the chosen viscosity exponent; VSS additionally changes the scattering-angle distribution. Read the generated reference-temperature, reference-cross-section and exponent conventions together: copying an exponent between definitions without its normalization changes the rate.

Use a collision database only within the declared temperature/energy range. A missing species pair, an out-of-range law and a zero physical rate are different states. The reusable comet collision database makes those distinctions explicit. Record the pair law, units, source and valid range in the run provenance rather than describing the model simply as collisions on.

8.3 No-time-counter selection and unequal weights

The NTC algorithm uses a majorant of σg and a rejection step. The majorant, trial normalization and accepted-particle update probabilities must describe the same weighted population. A diagnostic trial count by itself is not the physical collision count.

In the implemented general correction-aware NTC path, physical carrier $q_i = W_s c_i$ determines weighted selection. Cross-species trials use

$$N_{\mathrm{trial}} = \frac{(\sum_i q_i)(\sum_j q_j)(\sigma g)_{\max}\Delta t_{\max}}{q_{\min} V}. \tag{8.3}$$

Selected particles are updated with the prescribed $q_{\min}/q_i$ and local-timestep factors. Same-species self draws are discarded, not repeatedly redrawn with a different pair law. The simplified kernel is not a substitute: it explicitly rejects the unsupported correction-on configuration.

Dust is excluded before gas-species prefix sums are constructed. Otherwise a steep grain weight distribution can perturb gas pair selection even if no grain is ultimately collided. Reusable thread-local species buckets reduce allocation overhead without changing the intended traversal order.

8.4 Rotational and vibrational energy

For f_{rot} classical quadratic rotational degrees of freedom, equilibrium mean energy is $f_{\mathrm{rot}} k_B T/2$. Larsen–Borgnakke redistribution samples a microcanonical energy partition when the selected relaxation event occurs. The relaxation number Z_{rot} controls an event probability, not an energy multiplier: the maintained $Z = 2$ and $Z = 10$ profiles independently check probabilities $1/2$ and $1/10$.

For an ideal harmonic vibrational mode with characteristic temperature Θ_v, the thermal energy above its ground state is

$$\langle E_v \rangle = \frac{k_B \Theta_v}{\exp(\Theta_v/T) - 1}. \tag{8.4}$$

Use the production model's level convention and degeneracies. Equilibrium sampling, rotational relaxation, vibration–translation and vibration–vibration rates are separate capabilities; a rotational test does not establish all of them. Chemical formation energy must not be counted again as an internal relaxation source.

8.5 Independent validation

The core suites compare rates with analytic HS moments and independent Maxwell relative-speed quadrature for VHS. VSS checks angular moments as well as elastic invariants. Native particles in a 3-D AMR cell exercise actual collision and rotational-relaxation routines. Assess statistical error on accepted physical rates, relaxation times and equilibrium moments, and separately monitor conservation. A conservation-only test can pass a routine that performs no collisions at all.

Scope. The documented tests cover bounded selectors and profile combinations. They do not qualify every unequal-weight collision law, every species-dependent relaxation model or all VT/VV processes. Consult the test receipt and profile.

Source guide: `src/pic/README.md;test/srcCoreUnitTests/README.md;src/models/cometary_physics/README.md`

Chapter 9

Photochemistry and other reactions

9.1 Rates, channels and parent survival

For a constant first-order loss frequency β in s^{-1},

$$P_{\mathrm{survive}}(\Delta t) = e^{-\beta \Delta t}, \qquad P_{\mathrm{loss}} = 1 - e^{-\beta \Delta t}. \tag{9.1}$$

Compute small loss probabilities accurately, for example with `-expm1(-beta*dt)`. For time-varying illumination or rates, the exponent is $-\int \beta(t)\, dt$ along the actual trajectory. A constant-rate fixture cannot validate occultation crossings or a nonuniform radiation field.

For competing constant channels β_j, sample the event time from $\sum_j \beta_j$ and channel j with probability $\beta_j / \sum_k \beta_k$. An already committed event must not be repeated by another driver. Multiplicity belongs to stoichiometry: a channel producing two H atoms is not equivalent to a channel producing one H atom at twice its weight unless the chosen estimator explicitly proves that equivalence.

9.2 Daughters and event energetics

A parent-loss-only application deletes or changes the represented parent without allocating daughter particles. It must explicitly disclose that product transport and product conservation are outside its particle model. A daughter-producing application additionally owns species identity, product weights, insertion cells and velocity/internal-energy laws.

For a two-product channel with available translational energy E_{ex}, zero centre-of-mass momentum gives opposite momenta of magnitude

$$p_* = \sqrt{2 \mu E_{\mathrm{ex}}}, \qquad \mu = \frac{m_1 m_2}{m_1 + m_2}. \tag{9.2}$$

Transform products into the parent's inertial frame. Multibody products, electrons, ionization thresholds and internal excitation require their own partition laws; do not apply this two-body formula indiscriminately. Reaction tables must distinguish photon energy, threshold, recoil energy and energy already assigned to internal modes.

9.3 Charge exchange and electron impact

For incident particles in a background population, a typical charge-exchange frequency is $n_{\mathrm{bg}} \langle \sigma g \rangle$. Resonant exchange can swap charge labels while retaining the physical neutral/ion velocity roles; it is

not automatically removal of the neutral with no product. Electron-impact rates require the correct electron distribution, threshold and cross-section or rate fit. Units of $m^3\,s^{-1}$ and $cm^3\,s^{-1}$ must not be mixed.

The T6 native reaction owner checks optional charge-exchange and Burger electron-impact archives, branch identities and thresholds. That owner does not authorize those processes in an application whose input schema rejects plasma reactions. Rates inferred at a heliocentric distance or solar state must be converted once, with the selected scaling and provenance recorded.

9.4 Networks, stiffness and weighted estimators

The reusable reaction network expresses stoichiometry, channel rates and product multiplicities independently of a comet geometry. The chemical solver has separate analytic decay/kinetics, positivity and stiff-system tests. A deterministic abundance solver and stochastic particle chemistry solve related equations with different numerical errors. Neither is an independent reference if expected values are generated by the same implementation being graded.

The generic comet application supports population policies, correction windows, daughter packets, channel importance sampling and optional total-reaction forcing. These are unbiased-estimator requirements, not extra chemical channels. Preserve signed/residual estimator state through adaptation, migration and restart. Changing a proposal law requires an explicit physical- to-proposal weight ratio and independent rate/product-budget tests.

9.5 Transactional production callbacks

The ordinary photolysis callback remains available. An optional batched interface consists of begin, eligibility, batch-limit, reaction-processor and end-pass callbacks. All five must be installed together. Each event commits completely or leaves a deferred parent unchanged; new daughters are not eligible to be processed again in the same parent pass. The pre-exchange callback can create particle-buffer headroom before receive allocation. A partial callback set or an unsupported hybrid mode is rejected.

9.6 The Enceladus parent-loss scope

The maintained schema-2 example transports neutral H2O and CO2. The selected Burger water photo channels remove H2O parents; products are discarded. CO2 is transport-only in this receipt. No plasma background or daughter species may be inferred from the compiled species list. The exact ledger is

$$N_{\text{injected}} = N_{\text{photo}} + N_{\text{surface}} + N_{\text{escape}} + N_{\text{resident}} \tag{9.3}$$

per primary species, in the represented-particle convention of the run.

Source guide: `srcHartley2/CHEMISTRY.md;srcEnceladus/test/README.md;src/models/chemical_solver/README.md`

Chapter 10

Dust grains and grain populations

10.1 Size is a physical state

Dust is not a gas species with a larger fixed mass. Radius, material density, charge, velocity and any optical/material properties control its dynamics. For an ideal sphere of radius a and density ρ_d,

$$m_d = \frac{4\pi}{3}\rho_d a^3, \qquad \phi = \frac{q_d}{4\pi\epsilon_0 a}, \qquad \boldsymbol{a}_L = \frac{q_d}{m_d}(\boldsymbol{E} + \boldsymbol{v} \times \boldsymbol{B}). \tag{10.1}$$

The potential relation assumes an isolated spherical conductor. Shielding, nonspherical grains and material effects require different closures. The Lorentz term includes neither gas drag, gravity nor radiation pressure.

10.2 Size distributions and importance weights

For a number distribution $d\dot{N}/da = Ca^{-p}$ on $[a_{\min}, a_{\max}]$, total number and mass rates are different integrals:

$$\dot{N} = C \int a^{-p}\, da, \tag{10.2}$$

$$\dot{M} = \frac{4\pi\rho_d C}{3} \int a^{3-p}\, da. \tag{10.3}$$

The logarithmic limits $p = 1$ and $p = 4$ must be evaluated with their proper logarithms. A specified dust mass source does not determine a numerical particle weight until the size proposal and material density are given. For proposal density $g(a)$, an event's represented contribution contains the physical density divided by $g(a)$. This correction is independent of generic population splitting/merging.

The PIC-coupled dust model requires positive individual weight corrections. Logarithmic radius bins are half-open, with the global upper endpoint owned by the final bin. A steep number distribution can coexist with most of the injected mass in the largest grains. The in-coma size distribution additionally depends on size-dependent speed and survival; it need not reproduce the source distribution.

10.3 Charging and forces

Charging integrates currents appropriate to the declared plasma/illumination model. Validate equilibrium roots and transient convergence across multiple radii, including the sign convention for collection

and emission currents. A floating potential can be small while charge-to-mass acceleration is large. If plasma inputs are absent, an explicit neutral/uncharged model is required; zero-filled background data are not a physically justified charging solution.

Gas drag, radiation pressure, gravity and Lorentz forces have distinct validity regimes. Couple each force to the same grain state and frame. Radiation/optical interpolation tables need wavelength, radius, refractive- index convention and out-of-range policy. A Mie table lookup test is not validation of an entire dust tail.

10.4 Dust-specific split and merge

The dependency-light dust API groups particles by log radius and requires the same native owner cell, compiled species and size group. Current merging additionally requires exactly equal intrinsic radius, mass and charge. Same-bin membership alone is insufficient: two radii can have different charging and drag physics even if their difference looks small numerically.

Splitting clones one grain into n representatives with weights w/n. The supported merge maps at least three compatible particles to two: total weight W, weighted mean velocity $\boldsymbol{u}$, and velocities $\boldsymbol{u} \pm d\boldsymbol{e}_j$, each with weight $W/2$, where d^2 is the weighted velocity variance. It conserves represented number, mass, charge, momentum and scalar kinetic energy. It does not generally preserve the pressure tensor, higher moments or mean speed. The API can reject a requested sampled-speed constraint transactionally.

Scope. The dust split/merge physical kernel is implemented and tested. A general production PIC-list adapter is not supplied by that algebraic PASS. Generic gas split/merge must not be used to erase radius-dependent physics.

10.5 Spherical injection and its ledger

The production sphere injector chooses surface position relative to the actual centre, applies native ownership and local-time acceptance, creates the record and dispatches the selected mover. A signed residual mass reservoir accounts for finite particle granularity. A negative residual is not negative physical grain mass; it records that an event has advanced a fractional budget. The native fixture checks this ledger over two intervals and verifies that gas NTC leaves steep-weight grain records unchanged.

Source guide: `src/models/dust/README.md;test/srcModelDustTest/README.md;test/srcModelDustInjectionTest/README.md`

Chapter 11

Surfaces, arbitrary shapes and source boundaries

11.1 Geometry is separate from surface physics

Supported application geometries include analytic spherical boundaries and triangulated surfaces with cut-cell operations. Terrain-derived lunar meshes and comet shape models add data requirements. A valid triangle mesh does not specify temperature, composition, adsorption, sputtering yield or gravity. Those are explicit physical closures.

Let $\boldsymbol{n}$ point from the solid into the gas. An incident particle has $(\boldsymbol{v} - \boldsymbol{u}_{\text{wall}}) \cdot \boldsymbol{n} < 0$. Always state the surface-normal and wall-velocity conventions. For specular reflection,

$$\boldsymbol{v}' = \boldsymbol{v} - 2[(\boldsymbol{v} - \boldsymbol{u}_{\text{wall}}) \cdot \boldsymbol{n}]\boldsymbol{n}. \tag{11.1}$$

The invariant is kinetic energy in the local wall frame, not necessarily in the inertial frame. Moving walls can exchange physical work with particles.

11.2 Diffuse emission and accommodation

Fully diffuse thermal emission from a stationary surface at temperature T uses the outgoing flux distribution. With thermal tangential components, the positive normal component obeys

$$p(v_n) = \frac{m}{k_B T} v_n e^{-m v_n^2/(2k_B T)}, \qquad v_n > 0. \tag{11.2}$$

Thus $v_n = \sqrt{-2k_B T \log(1-u)/m}$. Sampling the positive half of an ordinary Maxwellian is not the same boundary flux law. For Knudsen/cosine angular emission over the hemisphere, $p(\mu) = 2\mu$, $0 \le \mu \le 1$, so $\mu = \sqrt{u}$; uniform solid angle instead has uniform μ. Isotropic surface position is another independent distribution. Use the selected law explicitly.

Accommodation, sticking, adsorption and desorption are separate probabilities or fluxes. Preserve a surface inventory balance,

$$N_{\text{surf}}^{n+1} = N_{\text{surf}}^{n} + N_{\text{ads}} - N_{\text{des}} + N_{\text{other}}, \tag{11.3}$$

with all terms using the same element area, time interval, species and represented weight. Emptying a flux buffer must not empty the physical surface reservoir.

11.3 Exosphere source processes

Applications can combine thermal desorption, photon-stimulated desorption, impact vaporization, sputtering, vertical or user-defined emission. Each has its own yield, energy/angular law, surface depletion and illumination dependence. A uniform sputtering rate fixture does not validate a plasma-dependent yield table. The native spherical exosphere owner compiles and exercises five bounded source modes with independent rate/inventory ledgers; external/background injection is a domain-boundary process with a different contract.

11.4 Importing an arbitrary shape

Before using a NASTRAN/triangle shape:

1. Declare coordinate units, body origin, orientation, epoch and external ID convention. Scale exactly once into the application frame.
2. Reject malformed connectivity, zero-area facets and invalid IDs. Inspect normals and winding. A closed body needs a consistent inside/outside definition; an open sheet needs an explicitly different physical treatment.
3. Verify bounds, known lengths, signed/enclosed volume where meaningful and surface area under controlled refinement.
4. Check rays, tangencies, shared edges, concavities and translated copies. Round-trip the actual format without assuming dense consecutive node IDs.
5. Resolve local geometric features with the transport/cut-cell mesh and establish area/volume and observable convergence.

Generic I/O tests use synthetic sparse IDs and known geometry. They do not certify an arbitrary production shape for watertightness, self-intersection, small features or surface-source mapping. Large concave shape integrations can be costly during initialization even before the first particle step.

11.5 Source positions and normalization

For a uniform triangle, two uniforms can be transformed to barycentric weights with $\sqrt{u}$; choosing two unconstrained barycentric coordinates biases the area. Choose triangles proportional to integrated physical emission, not by triangle count unless every element has equal rate. For a source shape $f_i \geq 0$ used with total Q, $Q_i = Q f_i / \sum_j f_j$; an absolute source file already supplies Q_i. Reject a contradictory additional total in absolute mode.

Enceladus uses projected surface jets/stripes rather than adopting all their unprojected empirical fitting coordinates as physical launch points. Moon terrain, spherical exosphere and arbitrary comet surfaces have separate source/thermal adapters. An unchanged source file is part of comparison provenance, especially if rates were fitted to the same observations.

Source guide: `test/srcCoreExosphereTests/README.md;test/srcCoreMeshTests/README.md;srcEncela dus/SOURCE_DISTRIBUTION_FILE.md`

Chapter 12

Planetary, satellite and comet applications

12.1 A common kinetic engine, different physical targets

Planetary applications choose their own body shape, rotation, gravity, illumination, surface inventories, plasma/radiation fields and observation operators. A species list or successful mesh initialization is not a complete physical model. Record the target, epoch, frame, source law and loss/network scope with every run. Use ephemerides and orientation consistently in injection, acceleration, surface interception and synthetic observations.

12.2 The Moon: exosphere, terrain and observation geometry

`srcMoon` combines spherical or terrain-based surface geometry with target-specific exosphere processes and diagnostics. Its generic runtime block is selected with `-input FILE`; omission preserves the historical analytic-sphere regression. The terrain path requires declared LOLA products, labels, resolution and output paths. Do not replace unavailable topography with a sphere without recording a different physical case.

Spatial targets distinguish the near-surface, far-field, terminator and polar regions. Quick, regression and full science receipts are different numerical campaigns. A completed coarse wiring run must retain its M1 SKIPPED/NOT QUALIFIED designation when convergence criteria were not exercised. The lunar unit/integration runners cover geometry, source/thermal state, environment timing and bounded products; inspect their actual per-case status. Na tail and limb observations additionally require instrument geometry, velocity-dependent radiation, line-of-sight weighting and the declared exposure average.

Source guide: `srcMoon/README.md;srcMoon/test/README.md;srcMoon/examples/README.md`

12.3 Enceladus: sources, parent loss and Cassini comparisons

`srcEnceladus` separates a fitted-source exporter from the forward kinetic application. The currently implemented distribution identifier is `tenishev2014_hybrid_8`: eight point jets and seven stripe components. Stripe vertices are projected vertically onto the declared 252100-m reference sphere. The

separate source file states geometry, kinematics, frame and whether strengths are absolute rates or a normalized shape.

Equal-strength E3/E5 receipts exist to start and exercise the forward code. They are not a recovered individual-source fit. The E3/E5 mass-44 normalizations are apparent CO2 and carry a documented impact-fragmentation bracket; they must not be reported as unqualified physical CO2 production. The schema-2 run transports neutral H2O/CO2 with selected H2O photolytic parent loss only, no daughter particles and no plasma.

Observation requests select INMS density/count products and supported UVIS occultations. Density, detector count rate and occultation transmission are different operators. Only compare like quantities with compatible response, units, frame and epoch. A trajectory outside the mesh has no local density; do not interpolate across that gap or label it zero. Resolve closest approach with actual flyby cells, and plot sampling uncertainty and data availability.

Source guide: `srcEnceladus/SOURCE_DISTRIBUTION_FILE.md;srcEnceladus/test/README.md`

12.4 Comets: radial and shape-based gas/dust models

`srcHartley2` retains its historical name but contains a generic comet layer with a DIM1 spherical radial backend, legacy DIM3 shape application, configurable chemistry/collisions, internal energy, cooling and brightness. DIM2 axisymmetric execution is reserved rather than implemented. I3 uses `input/i3.conf` for compiled capabilities and `input/i3.in` plus `-case N` for physical cases. Halley and Hartley decks use their documented workflows; do not assume every alias stages the shared parser identically.

The radial backend retains three-dimensional velocity/trajectory geometry while exploiting spherical symmetry for storage and sampling. It cannot represent an arbitrary asymmetric jet or shadowed nonspherical coma. For those cases use the appropriate DIM3 adapter with a qualified surface, thermal map and gravity model. Nucleus production and escaping/net production can differ because of returning particles; state which quantity is fitted.

The separate collisionless reference supplies a far-coma column correction for selected I3 post-processing. It is not an extension of the DSMC domain. Its columns, normalization, assumptions and missing radiation-pressure effects are documented independently. Comparison tables imported from COMET1D are historical evidence, not new runs of this checkout.

Source guide: `srcHartley2/README.md;srcHartley2/COLLISIONLESS_REFERENCE.md;srcHartley2/i3/COMPARISON.md`

12.5 Planetary energetic particles

`srcEarth` includes backward-characteristic cutoff/access and instantaneous spectrum products using declared static/quasi-static field snapshots. A backward access calculation and a forward Monte Carlo density calculation are different estimators. Field, spectrum, detector response and observer epoch must form one accepted configuration with consistent units. Classify unresolved/trapped/escaped characteristics explicitly; an unfinished trajectory must not silently count as excluded flux.

The coupled Step-10/11 contracts produce content-identified snapshots and product manifests. Their portable analytic/replay tests do not qualify all external-data or live SWMF release matrices. Magnetic-only instantaneous results do not imply electric acceleration or long-duration validity across

changing fields. Other planetary application directories require their own input/data and acceptance review before reuse.

Source guide: `srcEarth/README.md`

Chapter 13

Heliospheric and energetic-particle physics

13.1 Field-line and three-dimensional formulations

`srcSEP` transports particles along magnetic field lines embedded in three-dimensional space and contains particle–Alfvén-wave coupling. `srcSEP3D` supplies three-dimensional transport and background adapters. They share physical libraries and input vocabulary where supported, but a key accepted by one application need not be meaningful in the other. The schema table marks ownership and unsupported combinations.

The isotropic Parker transport equation, in a common convention, is

$$\frac{\partial f}{\partial t} = \nabla \cdot (\boldsymbol{\kappa} \nabla f) - (\boldsymbol{U} + \boldsymbol{v}_d) \cdot \nabla f + \frac{\nabla \cdot \boldsymbol{U}}{3} p \frac{\partial f}{\partial p} + Q. \tag{13.1}$$

Here f is phase-space density and p momentum magnitude. Flux and differential-energy spectra include additional Jacobians; a power index for $f(p)$ is not automatically an index for dN/dE. Focused transport additionally resolves pitch angle $\mu = \boldsymbol{v} \cdot \boldsymbol{b}/v$, streaming/focusing and pitch-angle diffusion. A diffusion approximation is not valid merely because a mean free path parameter was supplied.

13.2 Mean free path, diffusion and turbulence

For suitable gyrotropic scattering,

$$\kappa_{\parallel} = \frac{v^2}{8} \int_{-1}^{1} \frac{(1-\mu^2)^2}{D_{\mu\mu}} \, d\mu, \qquad \lambda_{\parallel} = \frac{3\kappa_{\parallel}}{v}. \tag{13.2}$$

Convergence near $D_{\mu\mu} = 0$, resonance gaps and end points requires an explicit model and quadrature policy. Perpendicular diffusion, field-line wandering and deterministic drifts are different operators. Check normalization, stochastic-calculus convention and any drift correction against the actual stochastic differential equation.

The SEP tree includes integrated and wave-number-resolved turbulence, advection/cascade and particle-wave terms. Their conservation and normalization require matching wave energy, resonance convention and particle weights. An uncoupled prescribed-turbulence test does not validate feedback. The older detailed derivations retained in this directory are supplementary reading; current public model contracts and test receipts determine support.

13.3 Background providers and physical state

An analytic Parker background, a PFSS/coronal ambient, a reduced front and a full spatial CME have different capabilities. Record epoch, frame, units, generation and validity interval for all sampled fields. Interpolation must not silently substitute quiet ambient after a rejected shock solution. Background advancement must not depend on particle allocation or source planning when the selected background contract is particle-independent.

A reduced-front provider supplies the prescribed geometry, undisturbed ambient reference and front-local immediate Rankine–Hugoniot limits. It does not own a spatial sheath/ejecta volume. A geometric arrival is not necessarily an accepted fast-shock arrival. Diagnostics must retain normal speed, upstream-relative inflow, fast Mach number, obliquity, compression and admissibility rather than imposing a compression floor.

13.4 CME implementation and qualification boundary

Shared coronal, SWCME, full-volume and reduced-front libraries have separate software registries. Full-volume BG3D-4/G1 remains unqualified at this edition. The selected resumed analytical lateral-material development retains the 3-D ambient, a field-threaded material contact and 1200-s startup corotation transition. Bounded transport/mesh/CT/contact/Cauchy kernels are tested; they do not constitute a production MHD runtime or a qualified regional map.

For a regular material map $\boldsymbol{X}(\boldsymbol{a}, t)$, $F = \partial \boldsymbol{X}/\partial \boldsymbol{a}$, $J = \det F > 0$, the adiabatic Cauchy relations are

$$\rho = \rho_0/J, \qquad \boldsymbol{B} = F\boldsymbol{B}_0/J, \qquad p = p_0 J^{-\gamma}. \tag{13.3}$$

They preserve material mass and flux for a compatible map; they do not select that map, supply shock matching, or close force/heating/work budgets. An isothermal expanding ambient needs explicit thermal work/heating beyond the adiabatic subset.

The application can deliberately reject a source/provider even when its library exists. In particular, current srcSEP3D documentation rejects the unreleased SWCME runtime selection while preserving those decks as typed negative fixtures. The reduced-front source has a bounded constant-weight mode; log-uniform momentum importance remains reserved until its individual weight correction is implemented and qualified. Read the application capability check before selecting a historical example.

Source guide: `srcSEP/README.md;srcSEP3D/README.md;src/models/sep_corona_swcme/README_CROSS_RAY.md`

Chapter 14

SWMF and external background coupling

14.1 Component lifecycle

SWMF builds model libraries into one framework executable. AMPS supplies PT (particle tracker) and PC (particle-in-cell) wrappers in `srcInterface`. The control module owns component registration, processor layout, sessions, execution/coupling schedules and framework restart requests. An AMPS standalone executable does not pass through the same outer driver.

The wrapper actions include VERSION, MPI, READ, CHECK, STDOUT, FILEOUT and GRID, followed by session initialization, run, restart and finalize methods. READ can occur multiple times before CHECK. Fields and mesh descriptions must not be published before their initialization stage. The current AMPS wrappers contain partly stubbed checking/output-routing actions; this manual does not promote them to a validated full implementation.

14.2 Communicators and processor layout

Use the communicator supplied by the framework. The Fortran handle is converted with `MPI_Comm_f2c`; collective calls must involve exactly that component's participating ranks. Overlapping component processor sets execute sequentially on shared ranks. Disjoint sets permit concurrent evolution, subject to data dependencies. Thread support and actual affinity must be checked separately from processor counts.

A schematic two-component layout, in a SWMF `PARAM.in`, is:

```
#COMPONENTMAP
GM  0  1  1
PT  2  3  1
```

It requires four ranks and compatible compiled components. This is a layout fragment, not a complete runnable physical case.

14.3 Parameter ownership and schedules

SWMF compile-time model selection uses its own `Config.pl -v=...`. Its `##BEGIN_COMP PT` block carries component parameters; standalone AMPS generic runtime input is a different grammar. Always validate against the actual SWMF `PARAM.XML` and component XML: some historical prose examples reverse positional arguments. For example, a one-way physical-time cadence uses:

```
#COUPLE1
GM          NameSource
PT          NameTarget
-1          DnCouple
10.0        DtCouple
```

This is a 10-s schedule, not a physical justification for that interval. Time-accurate components must respect supplied synchronization limits. Tight coupling additionally requires a supported master/slave timestep contract. Permitting a component to step past a coupling epoch changes temporal accuracy and must be declared.

Three clocks matter: iterations since process start, accumulated steps including restart history, and physical simulation time. Returning a limit to the scheduler without advancing a physical state to that epoch is not evidence of synchronization. Test this explicitly for nonintegral ratios of the AMPS step to coupling interval and for first-data readiness.

14.4 Exchange contracts

Each exchanged quantity needs a species/fluid mapping, units, frame, location (cell centre, corner, face or field-line vertex), timestamp and validity. Transform vectors, velocities and electric fields with the correct frame convention; rotating-frame velocity is not just a rotation of an inertial vector. Keep magnetic interpolation compatible with any required divergence or flux constraint.

Exchange a coherent state generation rather than mixing fresh density with old velocity. Verify owner and received-ghost values independently. A successful send count is not proof of correct physical data. Negative tests should reject missing variables, nonfinite values, stale epochs, mismatched layout and unsupported fluid counts before particles consume them.

14.5 File providers, SPICE and empirical models

Offline backgrounds need a versioned file format, dimensional identities, frame/epoch and explicit missing-domain policy. CDF/Kameleon, BATSRUS/BATL, ARMS, CCMC and SWMF are separate optional interfaces with different fixtures. Finding a library on disk does not qualify its adapter. SPICE supplies geometry, not plasma. Record exact kernel versions, load order, coverage and target/frame/aberration choices. Kernel readiness requires real state/orientation queries for the requested epoch.

14.6 Coupled testing and isolation

SWMF's legacy test Makefile serializes its top-level recipes and suppresses some command failures while retaining `.diff` artifacts. A wrapper runner must inspect expected outputs and classify missing

evidence as ERROR. Never concurrently reconfigure the same SWMF/AMPS build tree for independent tests. Run isolated runtime cases from frozen executables after serialized configuration/build, keeping each result separate before aggregation.

The AMPS global SWMF provider case remains SKIP without an independently registered exchange fixture. Synthetic ABI or replay tests are valuable but do not replace a live component-communicator and exchanged-field test.

Source guide: `srcInterface/PT_wrapper.f90;srcInterface/PC_wrapper.f90;srcInterface/amps2swmf.cpp`

Chapter 15

Sampling, output, observations and restart

15.1 Data lifetime and normalization

Accumulate raw represented moments into the collecting sample, then publish a completed window. Normalize once using the actual volume, exposure/time and sample measure. The completed buffer must remain immutable while output reads it. Clearing or swapping a buffer is a lifecycle operation, not a physical evolution of persistent background state.

For weighted velocity moments, temperature uses the bulk-subtracted tensor, not mean speed squared. Cells with zero weight have undefined bulk moments; write a defined empty/validity policy rather than a fabricated physical value. Use physical cut-cell volumes where required. Distinguish number density (m^{-3}), mass density ($\mathrm{kg\,m}^{-3}$), surface flux ($\mathrm{m}^{-2}\mathrm{s}^{-1}$) and integrated column (m^{-2}).

15.2 Application-defined node data and Tecplot

Register static/sampled storage and output callbacks before layout freezing. Output consists of variable names, interpolation and value printing in the same order. Interpolation can touch temporary nodes that have no associated AMR block: callbacks must not assume every printed node is owner-local. Populate owner state and exchange required ghosts before printing.

`outputMeshTECPLOT` is geometry-only. `outputMeshDataTECPLOT` and `OutputDistributedDataTECPLOT` have different boundary/parallel traversal contracts. A registered sphere does not automatically clip whole output bricks. Use the writer appropriate to the geometric product and verify variable order, units, finite records and owner/ghost agreement.

Full-volume ASCII output can dominate disk and runtime. Prefer a declared trajectory/ray product when it supplies the observable, retaining the same completed sampling window. Suppressing a visualization dump must not suppress physical accumulation or its conservation receipt.

15.3 Synthetic observations

For an optically thin emissivity with effective fluorescence rate g, brightness involves $\int ng\,ds$ with the chosen radiance convention and solid-angle conversion. Do not replace a velocity-dependent g with

a constant unless that approximation is part of the model. Occultation transmission instead uses optical depth $\tau_\lambda = \int n\sigma_\lambda \, ds$, with $I/I_0 = e^{-\tau_\lambda}$ in the simple absorption limit. Detector count rates additionally require response, binning, viewing direction and background treatment.

Evaluate spacecraft density at its actual ephemeris and epoch. Map all comparison inputs to the same frame. Distinguish missing observations, out-of-domain model points and physical zeros. Record source-fit provenance: agreement with data used to fit a source is not independent validation.

15.4 Persistent formats and restart identity

Particle and data-file formats carry explicit kind/layout identities, checksums and compatibility policy. Atomic publication avoids accepting half-written output; it does not make an incompatible layout valid. The global-weight particle restart layout remains revision 2; local-weight restart has a separate revision-3 identity including block data. Unversioned local-weight payloads are rejected.

Restart must preserve particles, weights, relevant time/RNG state, associated application data and any residual chemical/source estimators. An empty cell or block can still own physical state and must not be dropped solely because it has no particles. After local block-state restoration, publish owner values to ghosts before another face transfer. Application-owned structured bytes need a semantic layout fingerprint: an unchanged byte count alone does not identify a reaction-product ordering.

15.5 Continuation and archive practice

Verify both uninterrupted and split/restarted evolution with independent budgets and selected observables. A same-binary file round trip proves less than cross-decomposition continuation. A particle checksum alone does not test sample buffers, surface inventories or chemistry residual state. Do not interpret restart as a portable scientific interchange format.

Archive the exact executable hash, compile/runtime/source receipts, external data hashes, seed/layout, units/frames, sampling intervals and process command. Keep generated runs apart from maintained numerical reference fixtures. Reference updates require scientific review; a new output is never its own independent expected solution.

Source guide: `src/README.md;src/pic/PERSISTENT_FORMATS.md;src/test/tests/pic/persistence/README.md`

Chapter 16

Verification, application tests and nightly reporting

16.1 What the global procedure does

`src/test/run_tests.py` selects exact suites/cases from a manifest, resolves machine tools and prerequisites, runs individual owners and reconciles their typed results. Core and registered application streams share one report. The registry owns *what may run*; individual adapters own their mechanics and assertions; the component inventory owns the remaining coverage gaps. An installed optional tool is not evidence that its scientific interface works.

Tests form several layers: dependency-light equations/algorithms, configured native PIC applications, MPI ownership/halo/migration, application startup and evolving end-to-end cases, portability/backend checks and expensive scientific qualification. Select the layer required by the change. A parser/startup PASS cannot replace an evolving simulation, and a regression reference cannot replace a convergence or observational test.

The separately generated Makefile application regressions are documented in Chapter 17 and individually cataloged in Appendix C. The 100 generated legacy owners are separate from the modern 55-suite registry. Neither the global application's bounded replacement tests nor the 13-directory legacy governance audit implicitly executes every Table-controlled regression.

16.2 Required machine configuration

Every global invocation, including listing and coverage, requires `-config`. Profiles live under `test/qualification-machine`, outside the production source tree. Start from `template.json`; the checked-in `passion.json` is one host's actual choices, not a universal default. The profile records tools as argv vectors, output/work/data paths, MPI launcher/rank flag, resource limits and optional capabilities.

Use a base profile and a small site/path overlay instead of editing runners:

```
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --config /path/to/local-overlay.json --campaign smoke
```

The second path is a placeholder the operator must provide. Do not label unavailable SPICE, MPI, GPU, kernel or observation data as available just to remove a SKIP. Set work/output paths to writable

storage with sufficient memory and quota. Distinguish compiler argv from shell text.

16.3 Short, full and core-only commands

All following commands run from the AMPS root. Use a new output directory for each campaign.

```
PROFILE=test/qualification-machine/passion.json

# Discover exact cases, grouped by stream.
python3 -B src/test/run_tests.py --config "$PROFILE" --list

# Fast control/library smoke campaign.
python3 -B src/test/run_tests.py --config "$PROFILE" \
  --campaign smoke --output-dir test_output/manual-smoke

# Every registered CORE owner; exclude application streams.
python3 -B src/test/run_tests.py --config "$PROFILE" \
  --match 'CORE-*' --jobs 8 --build-jobs 16 \
  --output-dir test_output/manual-core

# All routine registered streams.
python3 -B src/test/run_tests.py --config "$PROFILE" \
  --all --streams 3 --jobs 2 \
  --output-dir test_output/manual-all

# Explicit expensive/portability/distributed/qualification union.
python3 -B src/test/run_tests.py --config "$PROFILE" \
  --full-validation --streams 3 --jobs 2 --build-jobs 16 \
  --output-dir test_output/manual-full
```

`-all` selects the full registry; do not combine it with `-match 'CORE-*'` expecting core-only selection. Full validation activates costly campaigns but retains missing prerequisites and unimplemented gates as typed nonpasses. It does not fabricate data or release a model.

16.4 Concurrency and native builds

`-streams N` limits active independent streams and defaults to 1. The current streams are core, application-srcEnceladus and application-srcMoon. `-jobs N` limits nonexclusive suites within each stream; `-build-jobs N` controls compilation where supported. These are three different resource dimensions. MPI ranks and OpenMP threads are additional multipliers and must fit the allocation.

The current checkout gate is stronger than a build mutex: an exclusive owner waits for all active readers and blocks other streams for its entire protected suite. This prevents another suite reading generated headers during reconfiguration. Thus requested stream concurrency is a maximum, not a promise that native owners overlap. The full-volume CME owner also runs its local binaries sequentially; raising global jobs does not parallelize them. Independent private-library owners can overlap when their declared isolation permits it.

16.5 Exact cases, reporting and failures

```
python3 -B src/test/run_tests.py --config "$PROFILE" \
```

```
  --case PHYS-3D-COLLISION-RATE-001 \
  --case PHYS-3D-PHOTODECAY-001 \
  --output-dir test_output/manual-two-cases
```

Each case must have one stable owner. Progress identifies the case before it runs. Some imported model owners execute their complete local registry before filtering reported rows; consult their README before treating a case selector as a quick execution filter.

Table 16.1: Result vocabulary.

State	Meaning
PASS	The exact selected assertion/contract completed successfully.
FAIL	Validly executed acceptance or governance criteria were not satisfied.
SKIP	Explicit prerequisite/campaign/support limitation; no successful execution is inferred.
ERROR	Build/setup/protocol/timeout or incomplete execution prevented a trustworthy physical assertion.

Preserve skipped and failed identities, not just totals. A crash after some markers cannot silently mark unreported cases PASS. The final global JSON and JUnit XML accompany per-stream JSON/logs, suite logs and artifacts. Use `amps-core-tests.json` for machine analysis and links to retained evidence, not a scraped terminal summary alone.

16.6 Independent oracles and qualification

Use analytic rates, manufactured fields/trajectories, independently integrated moments, conservation budgets and controlled refinement. Add a negative control proving the actual grader rejects a real defect. Do not generate the reference with the production function under test. Retain original failed attempts separately from corrected evidence.

The 2026-11-10 core snapshot has 1067 cases: 825 PASS, 1 FAIL, 240 SKIP and 1 ERROR; 50 suites: 43 PASS, 1 FAIL, 5 SKIP and 1 ERROR. All 785 original preintegration PASS identities remain PASS. The sole current FAIL is strict incomplete coverage; the ERROR is the original 180-s full production CME-refinement ceiling. BG3D-4/G1 and complete core coverage remain open. These are historical results, not a result of compiling this manual.

16.7 Adding a test

Provide an exact stable ID, owner, independent reference, deterministic setup, resource/isolation contract, all result states and meaningful retained logs. Update manifest/registry, component and source/-configuration traceability, the relevant README and the generated `src/test/TEST_CATALOG.md`. Agents must update that catalog when adding tests. Run:

```
python3 -B src/test/generate_test_catalog.py
python3 -B src/test/generate_test_catalog.py --check
python3 -B src/test/validate_traceability.py
```

Registration checks are governance evidence; they do not close physical coverage. A reference update is a reviewed model change, not a way to bless unexpected output.

Source guide: `src/test/README.md;src/test/TEST_CATALOG.md;src/test/STATUS.md`

Chapter 17

Individual application regression tests

17.1 The legacy application procedure and its authorities

AMPS also retains application regressions generated from `MakefileTest/Table` into `Makefile.test`. These tests configure an application, build a native executable, create a run directory, evolve it and compare selected output with maintained references. They complement the modern core/application registry of Chapter 16; they are not automatically imported into that Python runner.

The edition snapshot has 153 Table records, describing 150 distinct targets. Three mover names have alternative machine-specific declarations. The generated `HOSTNAME=passion` file contains 100 application owners; 50 other declared targets are absent from this host snapshot. An omitted record is not an executed SKIP. A generated target is not necessarily enabled for the current compiler, provisioned with data, or physically qualified.

Appendix C describes all 100 generated owners individually, with source line numbers, named purpose, actual comparison type, explicit tolerance overrides and detected prerequisites. It separately lists all 50 omitted names and their machine selectors. The companion `doc/legacy_test_catalog.json` retains exact phase locations and both source hashes. These counts are an editorial inventory, not a new test run.

17.2 How Table records control generation

The maintained record is delimited by `##>` and `<##`.

Table 17.1: Application-regression declaration fields.

Field	Meaning
`Name`	Application/test path. The generator removes the `test/` prefix from the target name. Example: `Name=test/fast-wave` produces `test_fast-wave`.
`Time`	Declared approximate runtime used by the historical scheduling tools; not a measured duration or an enforced physical acceptance limit.
`Type`	`amps` or `swmf`; distinguishes a standalone case from a custom framework-coupled campaign.

Field	Meaning
`Keys`	Configuration options, frequently selecting SPICE, background data, OpenMP or vectorization. Machine-qualified alternatives are possible.
`Outs`	Declared output basenames for the default comparison. When omitted, the default basename is `test_NAME`.
`Ref`	Compiler/machine-dependent reference suffix, for example a number in brackets inserted before the rank-specific reference suffix.
`ExeptionCode`	Historical spelling retained by the generator. A compiler-dependent SKIP can suppress aggregate execution. It is not a physical PASS and a direct target has its own actual behavior.
`CustomRefSolutionPaths`	Site/reference locations used by custom recipes where defined; inspect the effective recipe for actual use.
`Compile`, `Rundir`, `Run`, `Check`	Optional replacement recipes. Lines beginning with `>>` acquire a recipe TAB; `<<` preserves make-level text such as conditionals.

A missing machine selector applies everywhere. An inclusion selector such as `@@(0+passion+pleiades)` names permitted machines; an exclusion selector such as `@@(-pleiades)` excludes named machines. `@@(0+)` names no permitted host. Selection and precedence are implemented by the production generator; do not infer a case's effective options simply by concatenating every matching-looking Table line.

The current `Config.pl` installation and `-test-blocks=N` paths compile/run `utility/TestScripts/table_parser.cpp`. The older Perl `utility/TestScripts/BuildTest.pl` and template files are historical implementation references and differ in some behavior. Installation normally generates two execution blocks; changing the count regenerates the file. Do not edit only the generated Makefile and expect a change to survive the next generation. Keep the source declaration and effective recipe consistent.

17.3 Four phases and an individual invocation

An owner normally provides:

```
test_NAME
test_NAME_compile
test_NAME_rundir
test_NAME_run
test_NAME_check
```

The aggregate target calls those phases in order. The compile phase selects the application and libraries. Rundir usually recreates `run_test_NAME` and moves the executable into it. Run advances that frozen executable. Check evaluates the declared files and writes diff/log artifacts. Each phase needs its predecessor's exact state; an old run directory is not proof that the current binary ran.

For the two currently preflight-adapted regression examples:

```
# From the idle AMPS root; choose a launcher supported by the host.
make test_Moon MPIRUN='mpiexec -n 4'
make test_fast-wave MPIRUN='mpiexec -n 4'
```

Their compile recipes invoke `src/test/native/prepare_legacy_native.py`, then the established configuration and top-level -j16 build. Some other custom recipes hard-code MPI rank counts or run nested SWMF campaigns, so overriding `MPIRUN` does not universally change their resources. Read the individual run recipe.

The generated `test_help` target is intended to list these names. In this edition, executing `make test_help` returns a shell quoting error: some generated echo lines lack their closing quote. Use the complete local catalog and actual target definitions until that separate generator defect is repaired. The manual documents the defect; it does not report the help command as passing.

17.4 Application and numerical families

Table 17.2: The 100 currently generated application owners.

Family / owners	What the fixtures exercise
Planetary/comet/dust: 13	Saturn MOP; Moon baseline/cut-cell/restart; Enceladus and Mercury; CG gas/dust and tetrahedral variants; model dust; two Europa cases and Venus hot oxygen.
Heliospheric/coupled: 29	Parker equation, focused transport, Boris and field-line transport; 3-D prescribed or SWMF IMF; SC/IH field-line/node attachment and diffusion variants; ARMS background; BATS-R-US PIC, Europa, outer-heliosphere and dynamic-flux-rope couplings.
Fields/waves/solvers: 22	Linear solver and OpenMP variant, GMRES, uniform/nonuniform periodic boundaries, current-on/off light waves, reconnection, eleven fast-wave selector/vector/thread/rebalancing variants and constant-field boundary behavior.
Trajectory/mover: 12	Individual Lapenta2017, Markidis2010, Boris, relativistic Boris and guiding-centre paths; CCMC; corresponding mover applications and relativistic guiding-centre fixture.
Numerical/process: 10	Four legacy GoogleTest applications, NTC/MF collision distributions, two chemistry references, AMR interpolation through two adapters and CDAP diagnostic.
Geospace: 14	Gridless/mesh-backed and trajectory cutoff rigidity, vertical/hybrid variants, analytic dipole/T96 forward products, gridless directional density/flux fixtures and live SWMF/PT products.

The legacy Enceladus moment comparison is different from the modern 10000-step E3 source/observation owner. The legacy Moon baseline is different from terrain/science qualification. A modern global mapping/audit that mentions a legacy directory does not execute every Table application: `CORE-LEGACY-REGISTRY-AUDIT-001` is a separate 13-directory governance audit, not the 100-case regression procedure.

17.5 References, tolerances and comparison artifacts

The default recipe compares `run_test_NAME/PT/plots/OUT.dat` with a maintained file under `output/test_NAME`, using the selected reference suffix and the actual rank-specific `ref_npN` convention. The historical `output` can be a link to a shared reference tree. Preserve those references and record their identity before a run.

Custom checks can reduce rows, remove nondeterministic columns, compare trajectories/surfaces/volumes separately or import SWMF's own diff artifacts. The current catalog detects DiffNum in 93 owners, plain diff in three, and four custom checks with neither direct invocation. Those four include SWMF diff imports and the legacy unit-test stage/log check.

`share/Scripts/DiffNum.pl` uses its own default absolute/relative criteria unless a recipe overrides them. The appendix reproduces explicit `-a=` and `-r=` settings; it does not silently substitute the modern core suite's tolerances. Plain diff is a text equality check. Compiler- and rank-specific references must match the actual executable and layout. Do not assume stochastic particle records are rank-identical.

An empty final diff means agreement only if every prerequisite comparison actually ran and required files exist. Check each expected sub-diff and the final combined result, as well as process completion. Some default multi-output paths redirect repeatedly to one diff; a last empty comparison must not hide an earlier failure. Some custom recipes conditionally copy a framework result. Missing upstream evidence is not a successful comparison.

The base `test_all` recipe suppresses some child make failures and clears old diff files. The root Makefile also adds a modern core-nightly prerequisite to `test_all`, so the effective target can combine both procedures. Use the explicit Python selection for the modern registry and an explicit reviewed individual legacy target for a bounded regression. Never infer all tests passed solely from the outer make return code.

17.6 Dependencies, preflight and concurrency

Site paths in `Keys` can require CSPICE/kernels, background archives, Kameleon, GoogleTest or a separately installed SWMF tree. Vector/thread variants require matching compiler, CPU and OpenMP settings. The catalog flags detected prerequisites; it is not a complete environment solver. Some recipes also copy files produced by another test: Moon-restart explicitly requires particle restart files from the matching Moon baseline.

Many historical compile recipes remove `srcTemp`, use -j8, or switch applications without the current required root/idle/build-removal preflight. They must be reconciled before execution under the present policy. The shared native preflight helper is explicitly present in the current Moon and fast-wave recipes; its presence does not qualify every other target. Twenty-four generated owners contain Git instructions in custom recipes. The current Git-read-only policy does not permit executing those commands; documentation of them is not permission to clone, switch, pull or update.

Historical execution blocks are not the Python runner's `-streams` abstraction. In particular, `test_compile` and `test_run_threadN` do not establish a safe cross-stream native-build lock. Do not run two legacy configuring/compiling targets concurrently in one checkout. A future global adapter must serialize mutable build work, freeze executables/inputs, retain per-case logs and faithfully import all FAIL/SKIP/ERROR states before combining reports.

17.7 Maintaining this documentation

After changing either authoritative file, refresh the local snapshot:

```
python3 -B doc/update_legacy_test_catalog.py
python3 -B doc/update_legacy_test_catalog.py --check
make -C doc check
make PDF
```

This editorial generator only reads declarations and recipes; it never regenerates `Makefile.test` or executes its commands. Add a reviewed purpose description for any new generated owner. Preserve all declarations, including machine-omitted entries, and never turn catalog membership into a reported runtime PASS.

Source guide: `MakefileTest/Table;Makefile.test;utility/TestScripts/table_parser.cpp`

Chapter 18

Examples that are also end-to-end tests

18.1 How to interpret an example

Each example below names an existing test owner, actual production path, independent acceptance and retained output. Use the global machine profile and a fresh output directory. Native owners perform the required clean configuration/build sequence. These examples are not newly executed science runs merely because they appear in the manual.

18.2 A 3-D kinetic application: motion, collisions and parent decay

```
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --test CORE-PIC-NATIVE-001 \
  --output-dir test_output/example-3d-physics
```

The native core-test application allocates a small real 3-D AMR mesh, an analytic sphere and particle records. Production movers, NTC collisions, Larsen–Borgnakke relaxation and whole-mesh parent photodecay are exercised. The exact tests include `PHYS-3D-COLLISION-RATE-001`, `PHYS-3D-COLLISION-RELAXATION-001` and `PHYS-3D-PHOTODECAY-001`. Their oracles include analytic collision rates, equilibrium internal/translational moments and exponential survival. Mover cases additionally test uniform-field trajectories and surface events.

Inspect case results, native configure/build/run logs and independent numeric budgets in the output's artifact inventory. This fixture verifies coupled software operations, not a physical planetary atmosphere. Its default photodecay path does not claim daughter transport.

18.3 A daughter-producing photochemical mesh

```
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --test CORE-PHOTOCHEM-DAUGHTER-001 \
  --output-dir test_output/example-daughters
```

This separate native profile selects individual weights and H2O to OH+H whole-mesh photochemistry. It checks parent disappearance, product species, AMR insertion and physical weight/mass/energy

budgets using independent expected channel behavior. It is the appropriate example when daughter allocation is part of the contract; it must not be replaced by a parent-loss PASS from the preceding owner.

18.4 Surface sources and inventory in a native exosphere

```
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --test CORE-EXOSPHERE-SPHERE-NATIVE-001 \
  --output-dir test_output/example-exosphere
```

The one-rank sphere compiles five bounded source modes, invokes the production source dispatch and mover, checks native ownership/individual weights, and closes production, adsorption/desorption and inventory ledgers. Independent expected weights and surface updates grade the realized events. Source occurrence counts establish observability, not measured physical yield fits. The configured fixture explicitly pins and verifies no-SPICE mode.

18.5 Dust charging and physical injection

```
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --test CORE-DUST-NATIVE-001 \
  --output-dir test_output/example-dust-charge
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --test CORE-DUST-INJECTION-NATIVE-001 \
  --output-dir test_output/example-dust-source
```

The charging owner checks equilibrium/transient convergence at four radii. The injection owner allocates a 3-D mesh and translated sphere and exercises production source insertion, movement, individual-weight sampling and a signed two-interval mass reservoir. Its steep-weight dust records are also protected against the gas collision path. These are separate profiles and must not compile concurrently in one checkout. Algebraic dust split/merge and optical/size contracts are in `CORE-DUST-001`.

18.6 Adaptive geometry and actual MPI ownership

```
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --test CORE-MESH-NATIVE-001 \
  --output-dir test_output/example-mesh
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --test CORE-MESH-MPI-OWNERSHIP-001 \
  --output-dir test_output/example-mesh-mpi
```

The first owner builds DIM1/2/3 profiles and verifies tree topology, refine/coarsen transfer, real mesh-file round trips, interpolation and triangle/NASTRAN geometry against dyadic/analytic references. The MPI owner runs its declared 1/2/4-rank matrix and reads actual owner/received-ghost, migra-

tion, periodic/loss and restart-decomposition state. Rank-zero local assertions are not substituted for remote evidence.

18.7 Enceladus E3: a physical forward-model campaign

Start with the dependency-light and pre-MPI owners, then request the evolving case on a properly provisioned machine:

```
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --test APPLICATION-ENCELADUS-FORWARD-001 \
  --output-dir test_output/example-e3-model
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --test APPLICATION-ENCELADUS-NATIVE-STARTUP-001 \
  --output-dir test_output/example-e3-startup
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --config /path/to/e3-capability-overlay.json \
  --test APPLICATION-ENCELADUS-E3-001 \
  --output-dir test_output/example-e3-evolving
```

The overlay must explicitly enable the documented `enceladus-e3` capability. The extended owner retains 13 identities and otherwise SKIPs. The 10000-step run uses H2O/CO2 primaries and 1000-step sampling windows, with flyby refinement, no plasma and H2O parent loss only. Acceptance includes completion/injection, positive timestep, exact species conservation, finalization and trajectory Tecplot/PNG products, with actual flyby-cell resolution. The source geometry/strength receipt is fixed input, not a new fit performed by the forward test.

Compare only supported INMS/UVIS operators; distinguish apparent CO2 and missing/out-of-domain data. A density dip caused by underresolved closest approach must be addressed by the mesh study, not by changing fitted strengths.

18.8 Moon: regression and science qualification are separate

```
python3 -B src/test/run_tests.py \
  --config test/qualification-machine/passion.json \
  --test APPLICATION-MOON-REGRESSION-001 \
  --output-dir test_output/example-moon
```

This owner requires the `moon-spice` capability and real SPICE paths. It imports the declared U01–U26 and linked I11–I20 outcomes. Quick wiring, full mesh convergence and observational comparisons keep distinct acceptance. The M0 owner retains its explicit skipped identities until its multi-build workflow meets mandatory native preflight. Missing data or deferred science must remain visible. The separate legacy `make -j test_Moon` comparison has its own frozen numerical reference and scope.

18.9 Generic comet chemistry and brightness

The standalone application suite is:

```
# From the AMPS root; see the Stage-10 Git boundary below.
srcHartley2/tests/run_all_stage_tests.sh
```

It is not currently one of the registered application streams. Its portable mode authenticates maintained fixtures, builds focused programs and checks staged chemistry/policy/brightness contracts; optional fresh modes require a reviewed native executable. A portable replay PASS must not be called a new converged I3 or Halley run. Stage-10 selftests intentionally create synthetic commits in isolated AMPS-local scratch repositories; the current Git-read-only policy requires explicit authorization before invoking such Git-writing tests. The command above documents the entry point, not an exception to that policy.

I3 compiles from `input/i3.conf`; the actual physical cases are selected with `./amps -i input/i3.in -case 1`. Validate rates, transport, daughter weights and shell-chord brightness independently. Legacy Hartley DIM3 builds pass but its retained runtime did not finish cut-cell volumes before the smoke ceiling; the manual therefore claims no complete DIM3 runtime PASS.

18.10 Legacy electromagnetic wave regression

```
make test_fast-wave -j
```

The maintained legacy recipe applies native preflight and compares proton, electron and combined numerical diagnostics. Retained final-tree runs have empty difference files. Treat this as a regression for its declared wave configuration, complemented by manufactured operator/deposition tests; it does not qualify every electromagnetic boundary, plasma or accelerator.

18.11 Further Table-controlled application examples

The complete legacy procedure and exact per-target check locations are in Chapter 17 and Appendix C. Additional end-to-end examples include:

- `test_Moon-restart`: build the declared continuation and consume the matching `test_Moon` particle restart files. A previous unrelated Moon directory is not a valid dependency. Compare the continuation's declared output with its rank-specific reference.
- `test_CG-dust-test` and `test_CG-dust-test-tetra-cut-mesh`: compare gas/dust diagnostics and the declared shape/cut-cell variant. Their custom checks include reduced water and dust samples; a gas-only comparison does not validate the dust products.
- `test_Collision-Test-NTC` and `test_Collision-Test-MF`: compare the legacy distribution outputs for the two collision selectors with their own reference families. Those regressions complement the independent analytic collision-rate tests of the core suite.
- `test_Chemistry-Test`: compare the two maintained chemical kinetic output fixtures. A plain configuration/launch success is insufficient.
- `test_InterpolateAMR-AMPS-test`: compare both centre- and corner-based interpolation outputs with the maintained reference.
- `test_Geospace-Flux-Gridless`: compare directional/spatial and dipole density/flux fixture products; the OpenMP/MPI variant is a separate generated owner with its own declared resources.

These recipe interfaces remain available, but the many historical build steps, site paths and reference mutations need the preflight review described in the application-regression chapter before execution under current policy. No additional application run is claimed by documenting these examples.

Chapter 19

Extending applications and maintaining the code

19.1 Begin with a physical and software contract

State the target equation, units, frame, initial/boundary conditions, source normalization, valid parameter domain and output observable before installing a callback. Distinguish a scientific assumption from an implementation detail. If a required physical closure is unspecified, request that decision rather than embedding a guessed default.

Choose the owner of each responsibility. Generic algorithms belong in the core/shared model; target-specific geometry, fit coefficients and observation operators belong in applications. Keep application adapters small enough to review without reading all particle algorithms. Preserve the null/default path when adding an optional hook.

19.2 A practical new-application sequence

1. Start from a small maintained native test or the nearest application with the same geometry and storage needs. Do not copy unrelated physics.
2. Add an explicitly named compile-time deck under `input`; select its source directory, species, dimension, layouts and supported modules.
3. Implement a runtime owner using the generic grammar where appropriate, with unknown-key rejection and cross-field validation.
4. Register node/particle storage and callbacks before layout freezing. Build/allocate the mesh and initialize complete owner state.
5. Add sources with a physical-rate ledger, the correct event law and native ownership. Derive time steps and weights from the declared numerical policy and measure the realized values.
6. Install the mover/boundary/reaction callbacks with exact lifecycle and list-ownership rules. Complete exchange before dependent sampling/output.
7. Add independent model tests, a bounded native end-to-end case and negative setup controls before a costly science campaign.

8. Document exact commands, files, output interpretation, requirements and unsupported paths. Register application tests globally only when the adapter can truthfully import every owned result.

19.3 Comments and documentation

Explain equations, dimensional factors, sign conventions, approximations, algorithmic order, tolerances and typed failure behavior. A comment repeating the next assignment does not explain why the assignment is physically correct. Document owner/ghost and generation boundaries whenever state crosses MPI. Keep nearby READMEs current and update this manual when a public workflow or capability changes.

Public interfaces should validate inputs before mutating state. Prefer an explicit unsupported/invalid outcome to a plausible fallback value. Separate model rejection from missing-data and build failures. Never clamp an invalid physical parameter solely to force a downstream solver to continue. When numerical roundoff bounds are appropriate, derive and document them separately from physical acceptance tolerances.

19.4 Portability and reproducibility

Use configured argv-based compilers/launchers; avoid host-specific paths in test implementations. Keep site settings in the required machine profiles or established native configuration workflow. Test both packed/aligned particle layouts when touching record access; verify DIM1/2/3 when touching tree or face indexing. Exercise one-/multi-rank ownership and changed decomposition when touching exchange or persistence.

Compiler sanitizers cover only instrumented code. Linking an uninstrumented archive into a sanitized consumer does not instrument that archive. An optional CUDA compile probe does not qualify a device physics algorithm. Retain compiler flags, binary hashes, input/data fingerprints and exact commands with each qualification receipt.

19.5 Performance without changing the experiment

Profile initialization, transport, collisions, chemistry, exchange, sampling and output separately. Record mesh size, particle population and ranks along with wall time. CPU time in one rank can be synchronization wait rather than local physical work. Optimize allocation/search structure only after proving equivalent rate, random-draw and ownership behavior where those are required.

Do not reduce a refinement matrix, filter a shock branch or increase a frozen routine timeout and then report the original acceptance as passed. Keep performance diagnostics distinct from physical qualification. Write scalable observables and avoid unnecessary full-volume text output.

19.6 Source and reference preservation

Use read-only Git inspection under the current repository policy. Leave edits unstaged and uncommitted; do not alter repository metadata or Git control files. A historical document suggesting commits is not authorization to run them. Import only reviewed source/reference inputs; generated run logs, plots

and output trees are evidence rather than application source. Before every native rebuild, preserve the root/idle, generated-build removal, configuration and top-level -j16 sequence from Chapter 2.

Source guide: `src/README.md;src/test/README.md`

Chapter 20

Diagnostics and troubleshooting

20.1 Start from the earliest failed boundary

Keep the full command, working directory, resolved inputs, machine profile, compiler/runtime logs, executable identity and first error. A final missing output often follows an earlier failed configuration, mesh or physics stage. Do not rerun into the same directory and erase the failing evidence. An initialization-only exit and a finished physical run must have different reported meanings.

Table 20.1: Symptom-to-diagnosis guide.

Symptom	Check first
Different application is built	Config.pl selection, staged deck, generated source directory and actual compile commands. Do not edit `.amps.conf` manually.
Unresolved Fortran/SPICE symbols	Effective linkage, compatible compiler/MPI ABI, site libraries and explicit global SPICE mode. A component's SPICE-off key need not disable global CSPICE linkage.
SPICE time/frame error	Exact kernels, load order, leap-second coverage, target/frame and requested epoch. Test a real state/orientation query.
MPI fails before model startup	Launcher/rank options, socket/IPC permissions, allocation, ABI and environment. Report infrastructure ERROR, not physics FAIL.
Very long initialization	Count blocks/triangles, inspect cut-cell volume integration/search stacks, compiler optimization and I/O; verify whether any particle iteration has actually started.
Particle-buffer exhaustion	Source weights/counts, daughter multiplicity, split policy, event atomicity, receive-side headroom and ownership leaks.
Density wrong by a constant	Weight, accessible volume, time/sample normalization, SI conversion and gross/net source definition.
Dip near closest approach	Actual flyby cell sizes, surface/cut volume, trajectory frame/time, out-of-domain points and noise; do not fit away mesh bias.
One species missing in comparison	Injected/simulated/chemistry lists, output metadata, domain support and observation response/data availability.
Reaction rate changes after restart	Restored weights/time/RNG, network fingerprint, residual state and duplicate event processing.

Symptom	Check first
Only some ranks stall	Component communicator, collective ordering, owner migration, unfinished trajectory continuation and stale ghosts.
No concurrency with many jobs	Exclusive suite barriers, stream count, sequential imported binaries and global checkout gate.
Zero make exit but failed legacy test	Required diff files, nonempty differences, ignored recipe failures and missing/truncated outputs.
Many SKIPs	Capability/data prerequisites, campaign selection and genuinely unimplemented gates; do not convert them to PASS.

20.2 Native build identity and SPICE isolation

Inspect both requested and effective configuration. The native fixture helper checks saved selection, real make expansion and actual C++ commands for no-SPICE builds before MPI. Option order matters: a later undefine can remove an earlier define. A macro defined to zero is still defined to `##ifdef`. Use evidence from the compiled invocation, not substring matching in one configuration file.

20.3 Distinguish numerical error from missing physics

Conservation can coexist with wrong transport or no reactions; test the rate and shape of the solution. A high-resolution deterministic bias does not shrink with more random particles. A source fitted to observations can hide a wrong mesh or response. Compare independent units, limiting cases and discretizations before changing physical parameters.

For surface/source bugs, inspect event time, local normal, wall-frame velocity, represented weight and destination list together. For coupling bugs, inspect the physical epoch and entire generation, including received ghosts. For restart bugs, compare the first post-restart operation, not only the state written immediately after reading.

20.4 Reporting a reproducible issue

Provide the smallest input and test selector that still demonstrates the defect, with the full nonpass log path and resource requirements. Include whether it reproduces in one rank, whether a clean native build was used, and the intended reference equation/tolerance. Retain a failing regression before changing production code. A fix must preserve the physical gate or explicitly obtain a reviewed scientific change to it.

Source guide: `src/test/lib/README.md;src/test/GEOMETRY_PERFORMANCE_INVESTIGATION.md;src/test/STATUS.md`

Appendix A

Reference map, units and documentation authority

A.1 Conventions

Table A.1: Common dimensional quantities.

Quantity	Convention
$\boldsymbol{x}, a, h$	Position, grain radius, cell length: metres.
$t, \Delta t, \beta$	Time/step: seconds; first-order frequency: s^{-1}.
n, ρ, p	Number density: m^{-3}; mass density: $\mathrm{kg\,m}^{-3}$; pressure: Pa.
$\boldsymbol{v}, \boldsymbol{p}$	Velocity: $\mathrm{m\,s}^{-1}$; momentum: $\mathrm{kg\,m\,s}^{-1}$. Context distinguishes scalar pressure.
$\boldsymbol{E}, \boldsymbol{B}, q$	Electric field: $\mathrm{V\,m}^{-1}$; magnetic field: T; physical charge: C.
W, c	Base statistical multiplicity and positive individual correction; neither is a physical grain radius or particle mass.
$Q, \dot{M}$	Number production: s^{-1}; mass production: $\mathrm{kg\,s}^{-1}$.
σ, k	Cross section: m^2; binary rate coefficient: $\mathrm{m}^3\,\mathrm{s}^{-1}$.
$\boldsymbol{n}, \mu$	Surface normal: unit vector from solid to gas in the surface chapter; pitch-angle cosine in heliospheric transport.

Application keys/tables may explicitly use eV, km, AU, degrees or cgs. Convert at their owning boundary and record the convention; never infer units from a column's magnitude. A coordinate transformation must specify origin, handedness, axes, epoch and velocity convention.

A.2 Where to find the authoritative detail

Table A.2: Maintained detailed contracts.

Topic	Repository authority
Generic runtime grammar	`src/INPUT_FILE.md`; `src/general/input/amps_input.h`.
Node state and output ABI	`src/README.md`.

Topic	Repository authority
Mover, weighted NTC and hooks	src/pic/README.md.
Persistence envelope/layout	src/pic/PERSISTENT_FORMATS.md.
Global tests and extension rules	src/test/README.md, src/test/TEST_CATALOG.md, src/test/STATUS.md.
Machine portability	test/qualification-machine/README.md.
Arbitrary geometry	test/srcCoreMeshTests/README.md; src/meshAMR.
Dust	src/models/dust/README.md.
Moon	srcMoon/README.md; srcMoon/test/README.md.
Enceladus	srcEnceladus/SOURCE_DISTRIBUTION_FILE.md; srcEnceladus/test/README.md.
Generic comet and reference	srcHartley2/README.md, srcHartley2/CHEMISTRY.md, srcHartley2/COLLISIONLESS_REFERENCE.md.
SEP applications	srcSEP/README.md, srcSEP3D/README.md; their configuration and example guides.
Full-CME qualification	src/models/sep_corona_swcme/docs/CODEX_CME_PLAN.md; src/models/sep_corona_swcme/README_CROSS_RAY.md.

The manual is an entry point to these contracts, not a second implementation of their parsers. When source and prose disagree, preserve the discrepancy and verify the actual supported behavior before changing a scientific gate.

A.3 Supplementary legacy notes

This directory retains idf.tex, ecsim.tex, parker_eq.tex, focused_transport.tex, focused_transport1.tex, turbulence.tex and shock.tex. They contain longer historical derivations and research notes. They are not automatically included because their conventions, references and implementation claims need a separate editorial reconciliation. Core topics are covered in the maintained chapters of this edition; preserving these notes does not imply that every equation or optional mode is released.

The four former operator chapters are preserved under doc/legacy. Their CVS instructions, machine scripts and historical input syntax are not current run instructions. AMPS.bib is retained as a historical bibliographic source; this edition embeds its selected bibliography and requires no external AGU style file or BibTeX pass.

A.4 Maintaining the manual

Update a chapter whenever a public CLI, physics scope, required input or acceptance boundary changes. Add a source pointer and executable verification recipe. Refresh the local suite appendix with update_test_catalog.py after registry changes; it is a snapshot, not runtime evidence. Keep every \input and figure below this directory. Run structural checks, compile until references settle and inspect title, equations, tables, listings and representative body pages before publishing a new PDF.

Appendix B

Registered test-suite snapshot

Edition snapshot: 55 suites and 1174 individual cases. This table records ownership and intent, not execution status. Exact individual descriptions remain in `src/test/TEST_CATALOG.md`.

Registry identity: `b36dcc4629b3ffac9bf0363642d8c8167c63f66273a310b9f81e85d935dcecbe`.

Table B.1: All registered core and application test owners.

Suite / cases / isolation	Contract and runner
`CORE-HARNESS-001` 13 cases; nonexclusive	Test registry, layered configuration, selection, and result-protocol self-test `src/test/tests/harness/self/run_test.py`
`CORE-GENERAL-INPUT-001` 5 cases; nonexclusive	Generic runtime-input grammar, diagnostics, includes, and canonicalization `src/test/tests/general/input/run_test.py`
`CORE-GENERAL-CONTAINERS-001` 6 cases; nonexclusive	General array, bit-array, stack, linked-container, string, and 4-D index utilities `src/test/tests/general/containers/run_test.py`
`CORE-GENERAL-NUMERICS-001` 10 cases; nonexclusive	Rank-1 through rank-5 arrays, managed stack, special-function identities, and independent quadrature oracles `src/test/tests/general/numerics/run_test.py`
`CORE-GENERAL-RNG-001` 7 cases; nonexclusive	Reproducible streams, endpoint exclusion, uniform/normal/gamma moments, and isotropic directions `src/test/tests/general/rng/run_test.py`
`CORE-GENERAL-MPI-001` 5 cases; nonexclusive	Two-rank CMPI_channel transfers and MPI-derived independent RNG seeding `src/test/tests/general/mpi_channel/run_test.py`
`CORE-GENERAL-LOGGER-001` 4 cases; nonexclusive	Thread-safe diagnostic formatting, long lines, bounded rotation, durable flush, and fatal subprocess `src/test/tests/general/logger/run_test.py`

Suite / cases / isolation	Contract and runner
CORE-UNITS-001 5 cases; nonexclusive	Independent SI/CGS/normalized factors, round trips, dimensional identities, and invalid/unrepresentable scale checks src/test/tests/pic/units/run_test.py
CORE-ECSIM-OPERATORS-001 5 cases; exclusive	Native periodic ECSIM curl, grad-div, Laplacian, and curl-curl manufactured-solution convergence at orders 2/4/6/8 src/test/tests/pic/ecsim_operators/run_test.py
CORE-PIC-AUXILIARY-001 6 cases; nonexclusive	Ray-box clipping, constant/affine column quadrature, typed stopping-table interpolation, and background update ownership src/test/tests/pic/auxiliary/run_test.py
CORE-ECSIM-BOUNDARIES-001 4 cases; nonexclusive	Periodic, Dirichlet, homogeneous-Neumann and typed unsupported perfect-conductor ECSIM increment-row policy src/test/tests/pic/ecsim_boundaries/run_test.py
CORE-PIC-DEPOSITION-001 4 cases; nonexclusive	Production CIC corner weights, one-particle charge/current totals, exact segment continuity, and typed invalid domains src/test/tests/pic/deposition/run_test.py
CORE-PIC-FIELD-SOLVER-001 5 cases; nonexclusive	Independent manufactured Poisson residual/convergence and periodic Yee dispersion/energy gates src/test/tests/pic/field_solver/run_test.py
CORE-PIC-FIELD-LINE-001 6 cases; nonexclusive	Field-line straight/curved interpolation, unique exchange, weighted splitting, and conservative repartition src/test/tests/pic/field_line/run_test.py
CORE-PIC-CALLBACK-OUTPUT-001 4 cases; nonexclusive	Runtime and particle callback order/exactly-once dispatch plus deterministic trajectory base formatting src/test/tests/pic/callback_output/run_test.py
CORE-PIC-RESTART-001 8 cases; nonexclusive	Versioned restart metadata, round-trip/continuation, bounds, corruption, compatibility, atomic publication, and legacy policy src/test/tests/pic/persistence/run_test.py
CORE-PIC-DATAFILE-001 3 cases; nonexclusive	Versioned background fixture, changed decomposition consumption, and kind/layout compatibility src/test/tests/pic/persistence/run_test.py
CORE-PIC-OUTPUT-001 2 cases; nonexclusive	Tecplot trajectory variable order/units, finite records, and invalid output rejection src/test/tests/pic/persistence/run_test.py

Suite / cases / isolation	Contract and runner
CORE-SPECIES-RADIATION-001 4 cases; nonexclusive	Helium/sodium radiation-table reference points, inverse-square scaling, acceleration units, and invalid domains src/test/tests/species/radiation/run_test.py
CORE-INTERFACE-ABI-001 11 cases; nonexclusive	C/C++/Fortran ABI, Geopack coordinate/unit conventions, and typed optional-provider contracts src/test/tests/interface/run_test.py
CORE-CHEMICAL-SOLVER-001 7 cases; nonexclusive	Dependency-light adaptive implicit chemistry integration, network validation, analytic kinetics, stiff Robertson behavior, positivity, conservation, and typed failures src/test/tests/models/chemical_solver/run_test.py
CORE-COMET-LIBRARIES-001 5 cases; nonexclusive	Imported reaction network, collision database, cooling, fluorescence and reference-data contracts src/test/tests/models/comet_libraries/run_test.py
CORE-SURFACE-EXOSPHERE-001 8 cases; nonexclusive	Dependency-light spherical surface geometry, default exosphere surface state, source normalization, angular laws, and flux accounting src/test/tests/models/surface_exosphere/run_test.py
CORE-EXOSPHERE-SPHERE-NATIVE-001 1 cases; exclusive	Clean configured one-rank spherical exosphere source dispatch, AMR ownership, particle weight, surface interaction, inventory, and sampling-ledger qualification src/test/tests/models/exosphere_native/run_test.py
CORE-SPUTTERING-001 10 cases; nonexclusive	Ice sputtering tables, selectable emission-angle laws, source-rate accounting, product ledgers, and H2O/O2 speed importance weights src/test/tests/models/sputtering/run_test.py
CORE-DUST-001 8 cases; nonexclusive	Dependency-light dust size distributions, charging/force limits, spherical source laws, Mie interpolation, moments, and dust-owned size-group split/merge src/test/tests/models/dust/run_test.py
CORE-DUST-NATIVE-001 1 cases; exclusive	Clean configured test/model-dust build and one-rank equilibrium/transient charging convergence across four grain radii src/test/tests/models/dust_native/run_test.py

Suite / cases / isolation	Contract and runner
CORE-DUST-INJECTION-NATIVE-001 1 cases; exclusive	Clean configured DUST build and one-rank production spherical injection, AMR ownership, individual-weight, mover, sampling, and mass-ledger qualification src/test/tests/models/dust_injection_native/run_test.py
CORE-PIC-NATIVE-001 94 cases; exclusive	One configured native AMPS build covering generator/deck validation, particle buffers, global units, particle-weight/time-step contracts, and 3-D PIC physics src/test/tests/pic/native_core/run_test.py
CORE-PHOTOCHEM-DAUGHTER-001 1 cases; exclusive	Production whole-mesh photochemistry dispatch with individual-weight H2O to OH+H daughter insertion and independent physical budgets src/test/tests/pic/native_core/run_test.py
CORE-T6-REACTIONS-NATIVE-001 7 cases; exclusive	Configured optional charge-exchange and Burger electron-impact archives with analytic rates, thresholds, branching, product identity, conservation, and typed unavailable domains src/test/tests/pic/native_core/run_test.py
CORE-PIC-PERIODIC-001 1 cases; exclusive	Configured periodic AMR initialization, real/ghost pairing, particle wrapping, list transfer, and associated-data synchronization src/test/tests/pic/native_core/run_test.py
CORE-PIC-COLLISION-VHS-001 1 cases; exclusive	Generated VHS parameter tables, speed-dependent cross sections, and independent Maxwell relative-speed quadrature src/test/tests/pic/native_core/run_test.py
CORE-PIC-COLLISION-VSS-001 1 cases; exclusive	Generated VSS sigma/alpha parameters, analytical angular moments, and elastic invariants through the production selector dispatcher src/test/tests/pic/native_core/run_test.py
CORE-PIC-IDF-ROTZ-Z2-001 1 cases; exclusive	Generated LB Zrot=2 profile and independent one-half RT event probability src/test/tests/pic/native_core/run_test.py
CORE-PIC-IDF-ROTZ-Z10-001 1 cases; exclusive	Generated LB Zrot=10 profile and independent one-tenth RT event probability src/test/tests/pic/native_core/run_test.py
CORE-PIC-SPLIT-MERGE-001 15 cases; exclusive	Packed individual-weight particle layout and conservative Cartesian split/merge physical moments src/test/tests/pic/native_core/run_test.py
CORE-PIC-PBUFFER-LAYOUT-ALIGNED-FIXED-001 7 cases; exclusive	Aligned-16 fixed-weight particle-record offsets, clone/reuse policy, list integrity, and isolated fatal paths src/test/tests/pic/native_core/run_test.py

Suite / cases / isolation	Contract and runner
CORE-PIC-PBUFFER-LAYOUT-ALIGNED-INDIVIDUAL-001 13 cases; exclusive	Aligned-16 individual-weight particle layout plus independent split/merge regression profile src/test/tests/pic/native_core/run_test.py
CORE-PIC-WEIGHT-TIME-LOCAL-001 5 cases; exclusive	Generated species-local particle-weight and time-step profile using independent block-associated storage src/test/tests/pic/native_core/run_test.py
CORE-PIC-WEIGHT-TIME-SINGLE-GLOBAL-001 4 cases; exclusive	Generated single-global particle-weight and time-step profile proving cross-species slot aliasing src/test/tests/pic/native_core/run_test.py
CORE-MESH-MPI-OWNERSHIP-001 16 cases; exclusive	Collective 1/2/4-rank AMR ownership, owner-to-ghost halo, send-mask, block/particle migration, periodic/loss, restart-decomposition, and rank-invariant global-state matrix src/test/tests/pic/mpi_distributed/run_test.py
CORE-MESH-NATIVE-001 58 cases; exclusive	Three clean one-rank DIM profiles covering AMR topology/adaptation/data transfer, cut-cell geometry, NASTRAN I/O, and production PIC interpolation src/test/tests/mesh/native/run_test.py
CORE-MODEL-SEP-COMMON-001 143 cases; exclusive	All archive, mean-free-path, parallel/perpendicular-diffusion and pitch-angle local registries with explicit acceptance limitations src/test/tests/models/sep_common/run_test.py
CORE-MODEL-SWCME-001 147 cases; exclusive	Executable bounded R1 plus complete dormant extended and observational registries with typed blockers src/test/tests/models/swcme/run_test.py
CORE-MODEL-SEP-CORONAL-CME-001 222 cases; exclusive	Authoritative 222-case staged coronal-CME software-verification registry src/test/tests/models/cme/sep_coronal_cme/run_test.py
CORE-MODEL-SEP-CORONA-SWCME-FULL-001 46 cases; exclusive	Forty-six full-volume software/reference checks covering contact/slip, stable geometry, shared mesh/CT/energy and analytical material transport; BG3D-4/G1 remain unqualified src/test/tests/models/cme/sep_corona_swcme_full/run_test.py
CORE-MODEL-SEP-CORONA-SWCME-REDUCED-001 97 cases; exclusive	Complete 97-row reduced-front plus ambient registry with 35 explicit routine native SKIPs src/test/tests/models/cme/sep_corona_swcme_reduced/run_test.py

Suite / cases / isolation	Contract and runner
CORE-LEGACY-REGISTRY-AUDIT-001 13 cases; nonexclusive	Explicit governance/disposition audit for every roadmap legacy application without behavioral migration qualification src/test/tests/legacy/registry/run_test.py
CORE-T12-PORTABILITY-001 16 cases; nonexclusive	Bounded compiler, Fortran ABI, sanitizer, leak, OpenMP, MPI, CUDA and performance matrix plus the truthful strict-coverage gate src/test/tests/portability/matrix/run_test.py
APPLICATION-ENCELADUS-FORWARD-001 42 cases; nonexclusive	Complete dependency-light Enceladus forward-model parser, source, chemistry, numerics and 31-value reference campaign src/test/tests/applications/enceladus/run_test.py
APPLICATION-ENCELADUS-NATIVE-STARTUP-001 6 cases; exclusive	Clean Enceladus build and exact import of all six pre-MPI native startup/configuration gates src/test/tests/applications/enceladus_native/run_test.py
APPLICATION-ENCELADUS-E3-001 13 cases; exclusive	Capability-gated clean build and 10,000-step E3 process, transport-ledger, and flyby-product campaign src/test/tests/applications/enceladus_e3/run_test.py
APPLICATION-MOON-REGRESSION-001 36 cases; exclusive	Clean Moon/SPICE build plus all U01-U26 and bounded linked I11-I20 regression outcomes with typed scientific skips src/test/tests/applications/moon_regression/run_test.py
APPLICATION-MOON-M0-001 10 cases; nonexclusive	Complete Moon I01-I10 M0 registry with explicit non-execution until its multi-build workflow satisfies the mandatory native preflight src/test/tests/applications/moon_m0/run_test.py

Appendix C

Legacy application-regression catalog

153 Table records describe 150 distinct targets. The current generated Makefile contains 100 owners for host passion; 50 targets are absent from this host snapshot. Presence is not execution, support or PASS. Read Chapter 17 before executing a recipe. The local companion `legacy_test_catalog.json` records all phase line numbers, selectors and source hashes.

C.1 Heliospheric transport and coupled plasma

Table C.1: Generated targets: Heliospheric transport and coupled plasma.

Target and recipe location	Purpose and actual check contract
`test_SEP--Parker3D--SWMF_IMF` Table lines 635. Makefile.test compile/check lines 1866/1911.	Three-dimensional SEP transport with an SWMF-supplied IMF. 1 DiffNum call(s); default comparator tolerances. **Git instructions in recipe.** SWMF component tree.
`test_SEP--focused_transport-IH-mflampa-fl` Table lines 931. Makefile.test compile/check lines 1927/1973.	SWMF inner-heliosphere focused-transport configuration mflampa-fl. 1 DiffNum call(s); default comparator tolerances. **Git instructions in recipe.** SWMF component tree.
`test_SEP--focused_transport-IH-Kartavykh-drift` Table lines 1003. Makefile.test compile/check lines 1987/2033.	SWMF inner-heliosphere focused-transport configuration Kartavykh-drift. 1 DiffNum call(s); default comparator tolerances. **Git instructions in recipe.** SWMF component tree.
`test_SEP--focused_transport-IH-Kartavykh-drift_diffusion` Table lines 1073. Makefile.test compile/check lines 2049/2095.	SWMF inner-heliosphere focused-transport configuration Kartavykh-drift_diffusion. 1 DiffNum call(s); default comparator tolerances. **Git instructions in recipe.** SWMF component tree.

Target and recipe location	Purpose and actual check contract
`test_SEP--focused_transport-IH-Kartavykh` Table lines 1144. Makefile.test compile/check lines 2111/2157.	SWMF inner-heliosphere focused-transport configuration Kartavykh. 1 DiffNum call(s); default comparator tolerances. **Git instructions in recipe.** SWMF component tree.
`test_SEP--focused_transport-IH-He` Table lines 1214. Makefile.test compile/check lines 2173/2218.	SWMF inner-heliosphere focused-transport configuration He. 1 DiffNum call(s); default comparator tolerances. **Git instructions in recipe.** SWMF component tree.
`test_SEP--focused_transport-IH-He-drift` Table lines 1282. Makefile.test compile/check lines 2234/2279.	SWMF inner-heliosphere focused-transport configuration He-drift. 1 DiffNum call(s); default comparator tolerances. **Git instructions in recipe.** SWMF component tree.
`test_SEP--focused_transport-IH-He-drift_diffusion` Table lines 1351. Makefile.test compile/check lines 2295/2340.	SWMF inner-heliosphere focused-transport configuration He-drift_diffusion. 1 DiffNum call(s); default comparator tolerances. **Git instructions in recipe.** SWMF component tree.
`test_AMPS-BATSRUS-PIC-coupling` Table lines 5019. Makefile.test compile/check lines 2356/2390.	Live BATS-R-US/AMPS coupling configuration PIC-coupling. No DiffNum/plain-diff invocation detected; inspect custom check and process result. **Git instructions in recipe.** SWMF component tree; declared OpenMP setup.
`test_SEP_SWMF_test19_fl` Table lines 5103. Makefile.test compile/check lines 2418/2460.	SWMF test19 SEP coupling regression with field-line configuration. 1 DiffNum call(s); -a=2.0E-14. **Git instructions in recipe.** SWMF component tree.
`test_SEP_SWMF_test19` Table lines 5174. Makefile.test compile/check lines 2476/2513.	SWMF test19 SEP coupling regression. 1 DiffNum call(s); -a=2.0E-14. **Git instructions in recipe.** SWMF component tree.
`test_SEP-SC-IH--FL--FL-attachment--GC` Table lines 5237. Makefile.test compile/check lines 2527/2561.	Solar-corona/inner-heliosphere SEP attachment configuration FL–FL-attachment–GC. 4 DiffNum call(s); -a=2.0E-14. **Git instructions in recipe.** SWMF component tree.
`test_SEP-SC-IH--FL--FL-attachment--Droge` Table lines 5303. Makefile.test compile/check lines 2581/2627.	Solar-corona/inner-heliosphere SEP attachment configuration FL–FL-attachment–Droge. 4 DiffNum call(s); -a=2.0E-14. **Git instructions in recipe.** SWMF component tree.

Target and recipe location	Purpose and actual check contract
`test_SEP-SC-IH--FL--FL-attachment--Droge_diffusion` Table lines 5389. Makefile.test compile/check lines 2651/2697.	Solar-corona/inner-heliosphere SEP attachment configuration FL–FL-attachment–Droge_diffusion. 4 DiffNum call(s); -a=2.0E-14. **Git instructions in recipe.** SWMF component tree.
`test_SEP-SC-IH--FL--FL-attachment--He_diffusion` Table lines 5476. Makefile.test compile/check lines 2721/2768.	Solar-corona/inner-heliosphere SEP attachment configuration FL–FL-attachment–He_diffusion. 4 DiffNum call(s); -a=2.0E-14. **Git instructions in recipe.** SWMF component tree.
`test_SEP-SC-IH--FL--FL-attachment--Droge_diffusion_Borovikov` Table lines 5566. Makefile.test compile/check lines 2792/2838.	Solar-corona/inner-heliosphere SEP attachment configuration FL–FL-attachment–Droge_diffusion_Borovikov. 4 DiffNum call(s); -a=2.0E-14. **Git instructions in recipe.** SWMF component tree.
`test_SEP-SC-IH--FL--FL-attachment--Droge_diffusion_FTE` Table lines 5652. Makefile.test compile/check lines 2862/2908.	Solar-corona/inner-heliosphere SEP attachment configuration FL–FL-attachment–Droge_diffusion_FTE. 4 DiffNum call(s); -a=2.0E-14. **Git instructions in recipe.** SWMF component tree.
`test_SEP-SC-IH--FL--FL-attachment--Droge_diffusion_Parker` Table lines 5739. Makefile.test compile/check lines 2932/2978.	Solar-corona/inner-heliosphere SEP attachment configuration FL–FL-attachment–Droge_diffusion_Parker. 4 DiffNum call(s); -a=2.0E-14. **Git instructions in recipe.** SWMF component tree.
`test_SEP-SC-IH--FL--FL-attachment--Droge_diffusion_Qin` Table lines 5825. Makefile.test compile/check lines 3002/3048.	Solar-corona/inner-heliosphere SEP attachment configuration FL–FL-attachment–Droge_diffusion_Qin. 4 DiffNum call(s); -a=2.0E-14. **Git instructions in recipe.** SWMF component tree.
`test_SEP-SC-IH--FL--Node-attachment--GC` Table lines 5914. Makefile.test compile/check lines 3072/3108.	Solar-corona/inner-heliosphere SEP attachment configuration FL–Node-attachment–GC. 2 DiffNum call(s); -a=2.0E-14. **Git instructions in recipe.** SWMF component tree.
`test_AMPS-BATSRUS-Europa` Table lines 5974. Makefile.test compile/check lines 3123/3154.	Live BATS-R-US/AMPS coupling configuration Europa. No DiffNum/plain-diff invocation detected; inspect custom check and process result. **Git instructions in recipe.** SWMF component tree.

Target and recipe location	Purpose and actual check contract
`test_AMPS-BATSRUS-OH` Table lines 6029. Makefile.test compile/check lines 3170/3220.	Live BATS-R-US/AMPS coupling configuration OH. No DiffNum/plain-diff invocation detected; inspect custom check and process result. **Git instructions in recipe.** SWMF component tree.
`test_AMPS-BATSRUS_dynamic_fluxrope` Table lines 6111. Makefile.test compile/check lines 3236/3284.	Live BATS-R-US/AMPS coupling configuration dynamic_fluxrope. 1 DiffNum call(s); -a=2.0E-14. **Git instructions in recipe.** SWMF component tree.
`test_SEP--Parker_spiral--ParkerEq` Table lines 489. Makefile.test compile/check lines 3412/3432.	Prescribed Parker-spiral SEP transport; ParkerEq formulation. 3 DiffNum call(s); default comparator tolerances.
`test_SEP--Parker_spiral--FTE` Table lines 521. Makefile.test compile/check lines 3453/3473.	Prescribed Parker-spiral SEP transport; FTE formulation. 3 DiffNum call(s); default comparator tolerances.
`test_SEP--Parker_spiral--boris_relativistic` Table lines 554. Makefile.test compile/check lines 3494/3514.	Prescribed Parker-spiral SEP transport; boris_relativistic formulation. 1 DiffNum call(s); default comparator tolerances.
`test_SEP--Parker_spiral--field_line` Table lines 580. Makefile.test compile/check lines 3529/3549.	Prescribed Parker-spiral SEP transport; field_line formulation. 1 DiffNum call(s); default comparator tolerances.
`test_SEP--Parker3D--ParkerIMF` Table lines 606. Makefile.test compile/check lines 3564/3584.	Three-dimensional Parker-background SEP transport regression. 1 DiffNum call(s); default comparator tolerances.
`test_SEP3D-ARMS-coupling` Table lines 3500. Makefile.test compile/check lines 5099/5119.	SEP application with ARMS background data; compares the configured SEP3D diagnostic. 1 DiffNum call(s); default comparator tolerances. external background data.

C.2 Geospace cutoff, access and flux

Table C.2: Generated targets: Geospace cutoff, access and flux.

Target and recipe location	Purpose and actual check contract
`test_Geospace-SWMF-AMPS` Table lines 8168. Makefile.test compile/check lines 3298/3348.	Live SWMF geospace/PT setup followed by proton trajectory and volume-output comparisons. 2 DiffNum call(s); default comparator tolerances. **Git instructions in recipe.** SWMF component tree.
`test_Earth-cutoff-rigidity-gridless-steadystate_t96` Table lines 6353. Makefile.test compile/check lines 6198/6228.	Steady-state gridless cutoff-rigidity regression with T96 field. 2 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels.
`test_Earth-vertical-cutoff-rigidity-gridless` Table lines 6419. Makefile.test compile/check lines 6253/6283.	Gridless vertical cutoff-rigidity baseline. 5 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels.
`test_Earth-vertical-cutoff-rigidity-gridless-hybrid` Table lines 6517. Makefile.test compile/check lines 6335/6365.	MPI/OpenMP gridless vertical cutoff-rigidity baseline. 5 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels.
`test_Geospace-Flux-Gridless` Table lines 6616. Makefile.test compile/check lines 6417/6509.	Gridless geospace density/flux regression for directional, spatial and dipole fixtures. 20 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels.
`test_Geospace-Flux-Gridless-hybrid` Table lines 6932. Makefile.test compile/check lines 6675/6767.	MPI/OpenMP variant of the gridless density/flux fixture set. 20 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels.
`test_Earth-cutoff-rigidity-gridless-trajectory` Table lines 7250. Makefile.test compile/check lines 6933/7024.	Gridless cutoff-rigidity products along the declared trajectory. 8 DiffNum call(s); -r=1e-2. configured SPICE toolkit/kernels.
`test_Earth-vertical-cutoff-rigidity-gridless-trajectory` Table lines 7476. Makefile.test compile/check lines 7114/7205.	Gridless vertical cutoff along a spacecraft trajectory. 8 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels.

Target and recipe location	Purpose and actual check contract
`test_Earth-vertical-cutoff-rigidity-gridless-trajectory_dipole_only` Table lines 7713. Makefile.test compile/check lines 7307/7333.	Dipole-only gridless vertical cutoff trajectory fixture. 2 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels.
`test_Earth-cutoff-rigidity-3d` Table lines 7782. Makefile.test compile/check lines 7364/7394.	Mesh-backed three-dimensional cutoff-rigidity application regression. 4 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels.
`test_Earth-cutoff-rigidity-3d-analytic` Table lines 7873. Makefile.test compile/check lines 7440/7470.	Three-dimensional cutoff-rigidity analytic-fixture configuration. 4 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels.
`test_Earth-3d_forward-dipole_analytic` Table lines 7964. Makefile.test compile/check lines 7516/7542.	Forward three-dimensional analytic-dipole fixture; trajectory and volume products. 2 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels.
`test_Earth-3d_forward-t96_analytic` Table lines 8032. Makefile.test compile/check lines 7572/7598.	Forward three-dimensional T96/analytic-fixture configuration. 2 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels.
`test_Earth-3d_forward-t96` Table lines 8100. Makefile.test compile/check lines 7628/7654.	Forward three-dimensional T96 trajectory/volume regression. 2 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels.

C.3 Planetary, comet and dust applications

Table C.3: Generated targets: Planetary, comet and dust applications.

Target and recipe location	Purpose and actual check contract
`test_Saturn-MOP` Table lines 45. Makefile.test compile/check lines 3378/3397.	Saturn MOP water-population regression; reduced H2O mesh output. 1 DiffNum call(s); -a=4.0E-13. configured SPICE toolkit/kernels.
`test_Moon` Table lines 1529. Makefile.test compile/check lines 3744/3763.	Lunar exosphere particle-moment baseline and restart producer. 1 DiffNum call(s); default comparator tolerances. Native preflight helper present.

Target and recipe location	Purpose and actual check contract
`test_Moon-tetra-cut-cells` Table lines 1540. Makefile.test compile/check lines 3780/3799.	Lunar tetrahedral cut-cell mesh and particle-output regression. 2 DiffNum call(s); -a=4.0E-13.
`test_Moon-restart` Table lines 1565. Makefile.test compile/check lines 3818/3843.	Lunar continuation using particle restart files from test_Moon. 1 DiffNum call(s); default comparator tolerances.
`test_Enceladus` Table lines 1629. Makefile.test compile/check lines 3860/3879.	Legacy Enceladus application moment regression; distinct from the newer E3 forward campaign. 1 DiffNum call(s); default comparator tolerances. external background data.
`test_Mercury-test` Table lines 1685. Makefile.test compile/check lines 3896/3915.	Mercury application particle-output baseline. 1 DiffNum call(s); default comparator tolerances. configured SPICE toolkit/kernels; external background data.
`test_CG` Table lines 3724. Makefile.test compile/check lines 5134/5155.	Comet CG gas-coma application baseline. 1 DiffNum call(s); default comparator tolerances. external background data.
`test_CG-dust-test` Table lines 3757. Makefile.test compile/check lines 5172/5193.	Comet gas/dust regression including water and dust sample outputs. 3 DiffNum call(s); -a=4.0E-13. configured SPICE toolkit/kernels; external background data.
`test_CG-dust-test-tetra-cut-mesh` Table lines 3809. Makefile.test compile/check lines 5218/5239.	Comet gas/dust regression on the tetrahedral cut-cell mesh. 3 DiffNum call(s); -a=4.0E-13. configured SPICE toolkit/kernels; external background data.
`test_model-dust` Table lines 3860. Makefile.test compile/check lines 5264/5283.	Legacy dust charging/size model application regression. 1 DiffNum call(s); default comparator tolerances.
`test_Europa-test1` Table lines 4393. Makefile.test compile/check lines 5712/5731.	First maintained Europa source/particle regression configuration. 1 DiffNum call(s); default comparator tolerances. Kameleon and its data; external background data.
`test_Europa-test2` Table lines 4411. Makefile.test compile/check lines 5746/5768.	Second maintained Europa source/particle regression configuration. 2 DiffNum call(s); default comparator tolerances. Kameleon and its data; external background data.

Target and recipe location	Purpose and actual check contract
`test_Venus-Hot-Oxygen` Table lines 4662. Makefile.test compile/check lines 5923/5942.	Venus hot-oxygen application particle-moment regression. 1 DiffNum call(s); default comparator tolerances.

C.4 Numerical and process fixtures

Table C.4: Generated targets: Numerical and process fixtures.

Target and recipe location	Purpose and actual check contract
`test_unit_test` Table lines 1420. Makefile.test compile/check lines 3600/3623.	Legacy GoogleTest application; inspect assertions/exit and stage log, not an empty numeric diff alone. No DiffNum/plain-diff invocation detected; inspect custom check and process result. GoogleTest.
`test_unit_test_field-lines` Table lines 1446. Makefile.test compile/check lines 3636/3658.	Legacy GoogleTest field-line suite and summary-report comparison. 1 plain diff call(s). GoogleTest.
`test_unit_test_particle_distribu tion` Table lines 1472. Makefile.test compile/check lines 3672/3694.	Legacy GoogleTest particle-distribution suite and summary-report comparison. 1 plain diff call(s). GoogleTest.
`test_unit_test_collisions` Table lines 1499. Makefile.test compile/check lines 3708/3730.	Legacy GoogleTest collision suite and summary-report comparison. 1 plain diff call(s). GoogleTest.
`test_Collision-Test-NTC` Table lines 4525. Makefile.test compile/check lines 5788/5807.	No-time-counter collision fixture with multiple distribution/reference subchecks. 8 DiffNum call(s); default comparator tolerances.
`test_Collision-Test-MF` Table lines 4562. Makefile.test compile/check lines 5837/5856.	Majorant-frequency collision fixture with multiple distribution/reference subchecks. 8 DiffNum call(s); default comparator tolerances.
`test_Chemistry-Test` Table lines 4599. Makefile.test compile/check lines 5886/5905.	Chemical-solver application comparing two maintained kinetic reference outputs. 2 DiffNum call(s); default comparator tolerances.

Target and recipe location	Purpose and actual check contract
`test_InterpolateAMR-test` Table lines 4728. Makefile.test compile/check lines 5956/5980.	AMR interpolation reference values through the maintained test adapter. 1 DiffNum call(s); default comparator tolerances.
`test_InterpolateAMR-AMPS-test` Table lines 4760. Makefile.test compile/check lines 5994/6014.	Native AMPS centre- and corner-based AMR interpolation against one maintained reference. 2 DiffNum call(s); default comparator tolerances.
`test_cdap_test` Table lines 6184. Makefile.test compile/check lines 6033/6055.	CDAP diagnostic application and its configured numeric baseline. 1 DiffNum call(s); default comparator tolerances.

C.5 Fields, waves, solvers and periodic boundaries

Table C.5: Generated targets: Fields, waves, solvers and periodic boundaries.

Target and recipe location	Purpose and actual check contract
`test_linear-system-solver` Table lines 2132. Makefile.test compile/check lines 3932/3954.	Standalone linear-system solver diagnostic and reference-output regression. 1 DiffNum call(s); -a=1.0E-6.
`test_linear-system-solver-openmp` Table lines 2159. Makefile.test compile/check lines 3970/3994.	OpenMP linear-system solver variant with its reference diagnostic. 1 DiffNum call(s); -a=1.0E-6. declared OpenMP setup.
`test_periodic-bc-gmres` Table lines 2187. Makefile.test compile/check lines 4010/4032.	GMRES/periodic-boundary solver regression. 1 DiffNum call(s); -a=2.0E-7.
`test_lightwave-current-on` Table lines 2223. Makefile.test compile/check lines 4054/4076.	Electromagnetic light-wave regression with configuration current-on. 11 DiffNum call(s); -a=2.0E-14.
`test_magnetic-reconnection` Table lines 2359. Makefile.test compile/check lines 4124/4146.	Magnetic-reconnection diagnostic regression. 1 DiffNum call(s); -a=2.0E-14.
`test_magnetic-reconnection_avx256` Table lines 2450. Makefile.test compile/check lines 4168/4192.	Magnetic-reconnection diagnostic regression using AVX256. 1 DiffNum call(s); -a=2.0E-14. matching CPU/compiler vector target.

Target and recipe location	Purpose and actual check contract
`test_fast-wave` Table lines 2504. Makefile.test compile/check lines 4215/4237.	Electromagnetic fast-wave baseline regression, with proton/electron diagnostics. 2 DiffNum call(s); -a=2.0E-14. Native preflight helper present.
`test_fast-wave--legacy-ecsim` Table lines 2558. Makefile.test compile/check lines 4269/4291.	Electromagnetic fast-wave legacy ECSIM regression, with proton/electron diagnostics. 2 DiffNum call(s); -a=2.0E-14.
`test_fast-wave__correction` Table lines 2612. Makefile.test compile/check lines 4323/4345.	Electromagnetic fast-wave correction-enabled regression, with proton/electron diagnostics. 2 DiffNum call(s); -a=2.0E-14.
`test_fast-wave_rebalancing` Table lines 2666. Makefile.test compile/check lines 4377/4402.	Electromagnetic fast-wave load-rebalancing regression, with proton/electron diagnostics. 2 DiffNum call(s); -a=2.0E-14.
`test_fast-wave_AVX256` Table lines 2735. Makefile.test compile/check lines 4441/4465.	Electromagnetic fast-wave AVX256 regression, with proton/electron diagnostics. 2 DiffNum call(s); -a=2.0E-14. matching CPU/compiler vector target.
`test_fast-wave_AVX512` Table lines 2792. Makefile.test compile/check lines 4497/4521.	Electromagnetic fast-wave AVX512 regression, with proton/electron diagnostics. 2 DiffNum call(s); -a=2.0E-14. matching CPU/compiler vector target.
`test_fast-wave_multithread_Update JMassMatrix` Table lines 2850. Makefile.test compile/check lines 4553/4575.	Electromagnetic fast-wave threaded current/mass-matrix update regression, with proton/electron diagnostics. 2 DiffNum call(s); -a=2.0E-14.
`test_fast-wave_multithread_mover_ UpdateJMassMatrix` Table lines 2906. Makefile.test compile/check lines 4607/4629.	Electromagnetic fast-wave threaded mover and current/mass-matrix update regression, with proton/electron diagnostics. 2 DiffNum call(s); -a=2.0E-14.
`test_fast-wave_multithread_mover_ matmul_UpdateJMassMatrix` Table lines 2962. Makefile.test compile/check lines 4661/4683.	Electromagnetic fast-wave threaded mover/matrix multiply/current update regression, with proton/electron diagnostics. 2 DiffNum call(s); -a=2.0E-14.
`test_fast-wave-openmp` Table lines 3019. Makefile.test compile/check lines 4715/4739.	Electromagnetic fast-wave OpenMP regression, with proton/electron diagnostics. 2 DiffNum call(s); -a=2.0E-14. declared OpenMP setup.

Target and recipe location	Purpose and actual check contract
`test_fast-wave-openmp__shared_temp_list` Table lines 3076. Makefile.test compile/check lines 4771/4795.	Electromagnetic fast-wave OpenMP shared temporary particle list regression, with proton/electron diagnostics. 2 DiffNum call(s); -a=2.0E-14. declared OpenMP setup.
`test_lightwave-current-off` Table lines 3178. Makefile.test compile/check lines 4827/4849.	Electromagnetic light-wave regression with configuration current-off. 2 DiffNum call(s); -a=2.0E-14.
`test_lightwave-current-off-openmp` Table lines 3222. Makefile.test compile/check lines 4877/4901.	Electromagnetic light-wave regression with configuration current-off-openmp. 2 DiffNum call(s); -a=2.0E-14. declared OpenMP setup.
`test_Periodic-BC` Table lines 3270. Makefile.test compile/check lines 4929/4948.	Uniform periodic-boundary wave/particle outputs and tracked-trajectory regression. 9 DiffNum call(s); -a=4.0E-13.
`test_Periodic-BC-NonUniform` Table lines 3365. Makefile.test compile/check lines 5014/5033.	Nonuniform periodic-boundary wave/particle outputs and tracked trajectories. 9 DiffNum call(s); -a=4.0E-13.
`test_const-field-bc` Table lines 6207. Makefile.test compile/check lines 6072/6134.	Constant-field boundary application; compares its declared numerical boundary products. 6 DiffNum call(s); default comparator tolerances.

C.6 Individual trajectories and mover applications

Table C.6: Generated targets: Individual trajectories and mover applications.

Target and recipe location	Purpose and actual check contract
`test_Individual_Trajectories--Lapenta2017` Table lines 3867. Makefile.test compile/check lines 5300/5319.	Charged-particle Lapenta2017 trajectory/mover fixture. 1 DiffNum call(s); -r=2.0e-3. Kameleon and its data; external background data.
`test_Individual_Trajectories--Markidis2010` Table lines 3904. Makefile.test compile/check lines 5333/5352.	Charged-particle Markidis2010 trajectory/mover fixture. 1 DiffNum call(s); -r=2.0e-2. Kameleon and its data; external background data.

Target and recipe location	Purpose and actual check contract
`test_Individual_Trajectories--Boris` Table lines 3940. Makefile.test compile/check lines 5366/5385.	Charged-particle Boris trajectory/mover fixture. 1 DiffNum call(s); -r=2.0e-3. Kameleon and its data; external background data.
`test_Individual_Trajectories--Boris-relativistic` Table lines 3975. Makefile.test compile/check lines 5399/5418.	Charged-particle Boris-relativistic trajectory/mover fixture. 1 DiffNum call(s); -r=2.0e-3. Kameleon and its data; external background data.
`test_Individual_Trajectories--guiding-center` Table lines 4010. Makefile.test compile/check lines 5432/5451.	Charged-particle guiding-center trajectory/mover fixture. 1 DiffNum call(s); -r=2.0e-2. Kameleon and its data; external background data.
`test_CCMC-Individual_Trajectories` Table lines 4079. Makefile.test compile/check lines 5465/5484.	CCMC individual charged-particle trajectory regression. 2 DiffNum call(s); default comparator tolerances. Kameleon and its data; external background data.
`test_Mover-guiding-center` Table lines 4154, 4180. Makefile.test compile/check lines 5501/5521.	Charged-particle guiding-center trajectory/mover fixture. 1 DiffNum call(s); -r=2.0e-5. external background data.
`test_Mover-Lapenta2017` Table lines 4208. Makefile.test compile/check lines 5536/5556.	Charged-particle Lapenta2017 trajectory/mover fixture. 1 DiffNum call(s); default comparator tolerances. external background data.
`test_Mover-Markidis2010` Table lines 4237. Makefile.test compile/check lines 5571/5591.	Charged-particle Markidis2010 trajectory/mover fixture. 1 DiffNum call(s); default comparator tolerances. external background data.
`test_Mover-Boris` Table lines 4268, 4291. Makefile.test compile/check lines 5606/5626.	Charged-particle Boris trajectory/mover fixture. 1 DiffNum call(s); default comparator tolerances. external background data.
`test_Mover-Boris-relativistic` Table lines 4318, 4341. Makefile.test compile/check lines 5641/5661.	Charged-particle Boris-relativistic trajectory/mover fixture. 1 DiffNum call(s); default comparator tolerances. external background data.
`test_Mover-gca-relativistic` Table lines 4366. Makefile.test compile/check lines 5676/5696.	Relativistic guiding-centre mover application baseline. 1 DiffNum call(s); default comparator tolerances.

C.7 Declared but absent from the generated host snapshot

These are not executed SKIPs: they are omitted targets. Filters are shown as declared, without pretending this editorial tool implements the complete production generator.

Table C.7: Additional Table declarations and machine selectors.

Target	Declaration and selector
test_Mars-ions	Table line 77: (0+)
test_Mercury-ions	Table line 89: (0+)
test_Titan	Table line 117: (0+)
test_COPS	Table line 127: (0+)
test_CG-Rosina-Liouville--Trajectory	Table line 176: (0+)
test_CG-Rosina-Liouville	Table line 240: (0+)
test_SC-Cont	Table line 295: (0+)
test_test-1853-shape	Table line 355: (0+)
test_X-Wing	Table line 383: (0+)
test_CCMC	Table line 445: (0+)
test_SEP--Multiple_Parker_spiral--boris_relativistic	Table line 463: (0+)
test_SEP--focused_transport-IH-mflampa	Table line 709: (0+)
test_SEP-CME--focused_transport--mflampa-fl	Table line 779: (0+pleiades)
test_SEP-CME--focused_transport--fl-7au-droge	Table line 830: (0+pleiades)
test_SEP-CME--focused_transport--3D	Table line 881: (0+pleiades)
test_ices-reader	Table line 1580: (0+)
test_tecplot-reader	Table line 1590: (0+)
test_binary-reader	Table line 1596: (0+)
test_kameleon-reader	Table line 1619: (0+)
test_Rosetta	Table line 1635: (0+)
test_OH	Table line 1652: (0+)
test_Tempel1	Table line 1671: (0+)
test_batl-reader	Table line 1697: (0+)
test_Earth	Table line 1736: (0+)
test_Earth-cutoff-rigidity	Table line 1795: (0+)
test_Earth-cutoff-rigidity_t05	Table line 1868: (0+)
test_Earth-cutoff-rigidity-points_t05	Table line 1912: (0+)
test_Earth-geo_t05	Table line 1971: (0+)
test_Earth-cutoff-rigidity_t96	Table line 2015: (0+)
test_Earth-T96	Table line 2052: (0+)
test_Earth-Impulse-Source	Table line 2090: (0+)
test_Earth-Impulse-Neutron-Source	Table line 2114: (0+)
test_lightwave-current-on-openmp	Table line 2305: (0+)
test_magnetic-reconnection-openmp	Table line 2410: (0+)
test_photon-test	Table line 3134: (0+)
test_photon-test-openmp	Table line 3151: (0+)

Target	Declaration and selector
test_Legacy_2dCG	Table line 3460: (0+)
test_CG-PostProcess--Read-Trajectories--on	Table line 3529: (0+)
test_CG-PostProcess--Read-Trajectories--off	Table line 3597: (0+)
test_CG-VIRTIS-Inversion-Legacy	Table line 3649: (0+)
test_CG-DFMS-Inversion-Legacy	Table line 3687: (0+)
test_Individual_Trajectories--guiding-center--KAMELEON	Table line 4046: (0+valeriy)
test_CCMC-Individual_Trajectories--KAMELEON	Table line 4098: (0+valeriy)
test_CCMC-Individual_Trajectories--KAMELEON-multifile	Table line 4132: (0+valeriy)
test_LOS	Table line 4446: (0+)
test_Mars-Hot-Oxygen	Table line 4618: (0+)
test_Mars-Hot-Carbon	Table line 4689: (0+)
test_Drag-Coefficient--Specular-Reflection	Table line 4786: (0+)
test_Sphere-drag-coefficient	Table line 4839: (0+)
test_CG-BATSRUS-dust-coupling	Table line 4930: (0+)

Bibliography

[1] G. A. Bird, *Molecular Gas Dynamics and the Direct Simulation of Gas Flows*, Oxford University Press, 1994. https://doi.org/10.1093/oso/9780198561958.001.0001.

[2] V. Tenishev, M. Combi and B. Davidsson, A global kinetic model for cometary comae: the evolution of the coma of the Rosetta target comet Churyumov–Gerasimenko throughout the mission, *Astrophysical Journal*, 685, 659–677, 2008. Bibliographic provenance: the retained `AMPS.bib`.

[3] V. Tenishev, M. R. Combi and M. Rubin, Numerical simulation of dust in a cometary coma: application to comet 67P/Churyumov–Gerasimenko, *Astrophysical Journal*, 732(2), 2011. Bibliographic provenance: the retained `AMPS.bib`.

[4] AMPS development team, maintained source, input, physics and test contracts, repository edition of 10 November 2026. See Appendix A for module-specific paths and Chapter 16 for actual evidence.

[5] Center for Space Environment Modeling, *Space Weather Modeling Framework User Manual, Maintenance Manual, Testing Procedures and Software Standard*, local source edition reviewed for this manual. SWMF uses separate model wrappers and couplers; some older requirements/examples require reconciliation with its current `PARAM.XML` and implementation.